CA-01: Registro y gestión de cuentas bancarias

Requisito del documento oficial

Criterio: El sistema debe exponer un endpoint REST (**POST /api/cuentas**) que permita registrar cuentas bancarias con los campos: número de cuenta (único, 10 dígitos), titular, tipo de cuenta (ahorro/corriente), saldo inicial (≥ 0) y estado (activa/inactiva). Debe validar que no existan duplicados por número de cuenta y retornar la cuenta creada con su identificador.

Adicionalmente, exponer un endpoint:

- **GET /api/cuentas** para listar todas las cuentas registradas.

Respuesta al criterio

Se documenta el registro de una cuenta con HTTP 201 e identificador asignado, la consulta paginada con HTTP 200 y la persistencia en SQL Server. Las pruebas negativas rechazan número repetido (409), saldo inicial negativo (400) y número de cuenta de longitud distinta de diez dígitos (400).

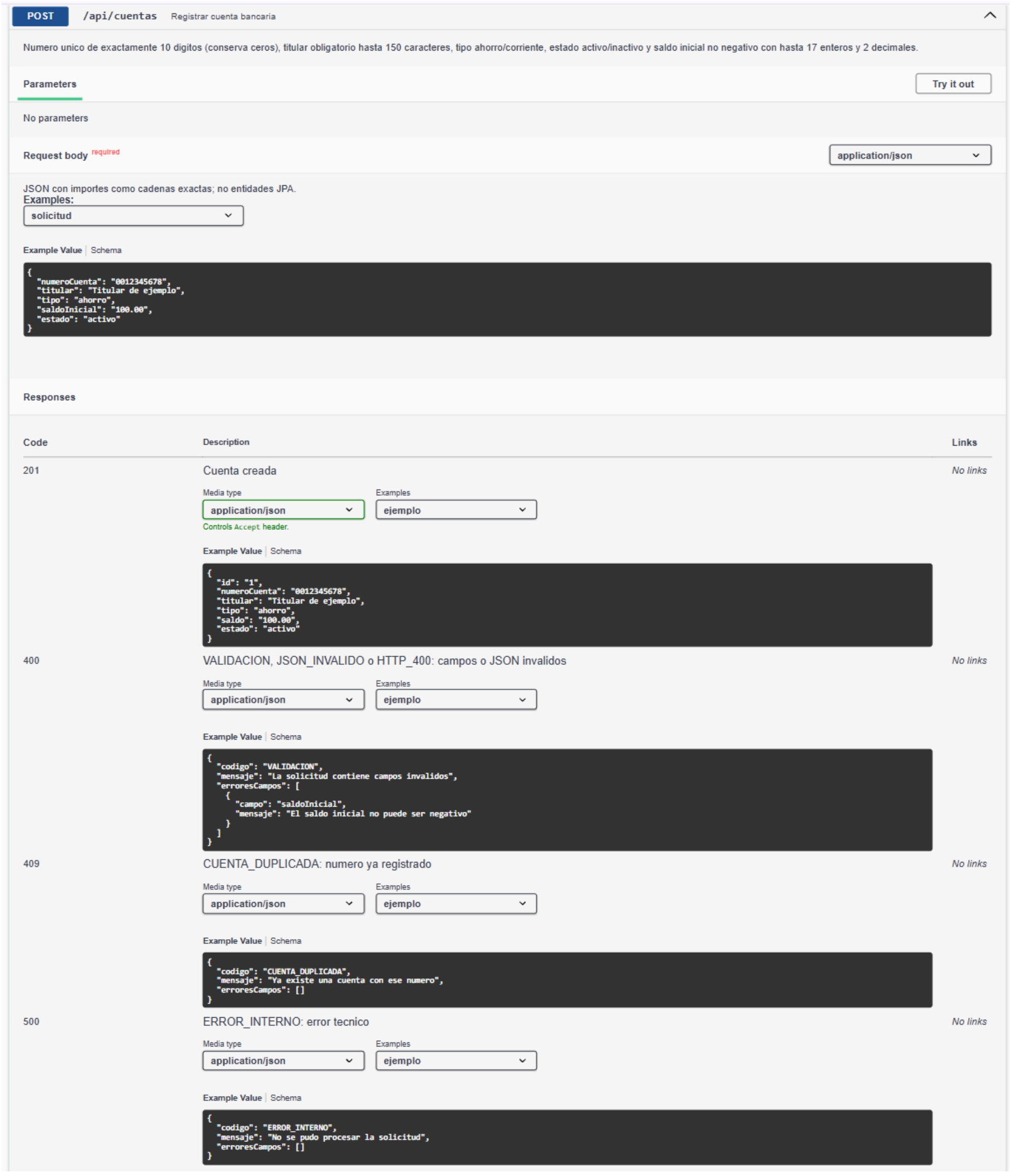
Capturas que acompañan la respuesta

1. POST api cuentas
2. Tamaño de digitos 10
3. Creacion de cuenta unica
4. Nr cuenta repetida
5. Saldo Negativo
6. Persistencia de cuentas
7. GET api cuentas

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-01: Registro y gestión de cuentas bancarias

Captura 1. POST api cuentas



Contrato del POST de cuentas: campos de entrada, ejemplos y códigos de respuesta.

CA-01: Registro y gestión de cuentas bancarias

Captura 2. Tamaño de digitos 10

JSON con importes como cadenas exactas; no entidades JPA.

Examples:

[Modified value]

```
{
  "numeroCuenta": "001234",
  "titular": "Titular de ejemplo",
  "tipo": "ahorro",
  "saldoInicial": "100.00",
  "estado": "activo"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/cuentas' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "numeroCuenta": "001234",
  "titular": "Titular de ejemplo",
  "tipo": "ahorro",
  "saldoInicial": "100.00",
  "estado": "activo"
}'
```

Request URL

```
http://localhost:8080/api/cuentas
```

Server response

Code	Details
400	Error: Bad Request

Response body

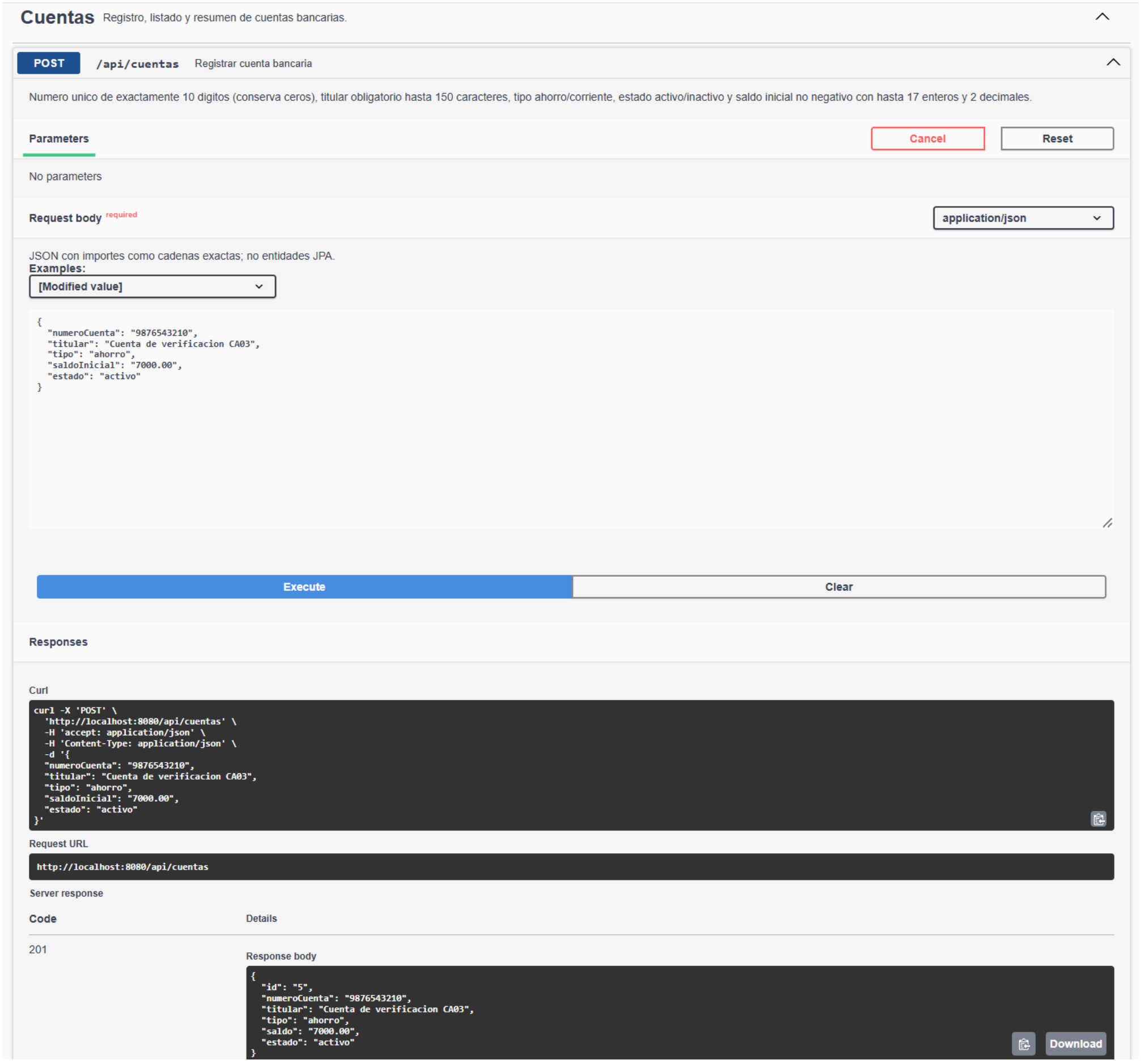
```
{
  "codigo": "VALIDACION",
  "mensaje": "La solicitud contiene campos invalidos",
  "erroresCampos": [
    {
      "campo": "numeroCuenta",
      "mensaje": "El numero de cuenta debe tener exactamente 10 digitos"
    }
  ]
}
```

Download

Número 001234: HTTP 400; se exigen exactamente diez dígitos.

CA-01: Registro y gestión de cuentas bancarias

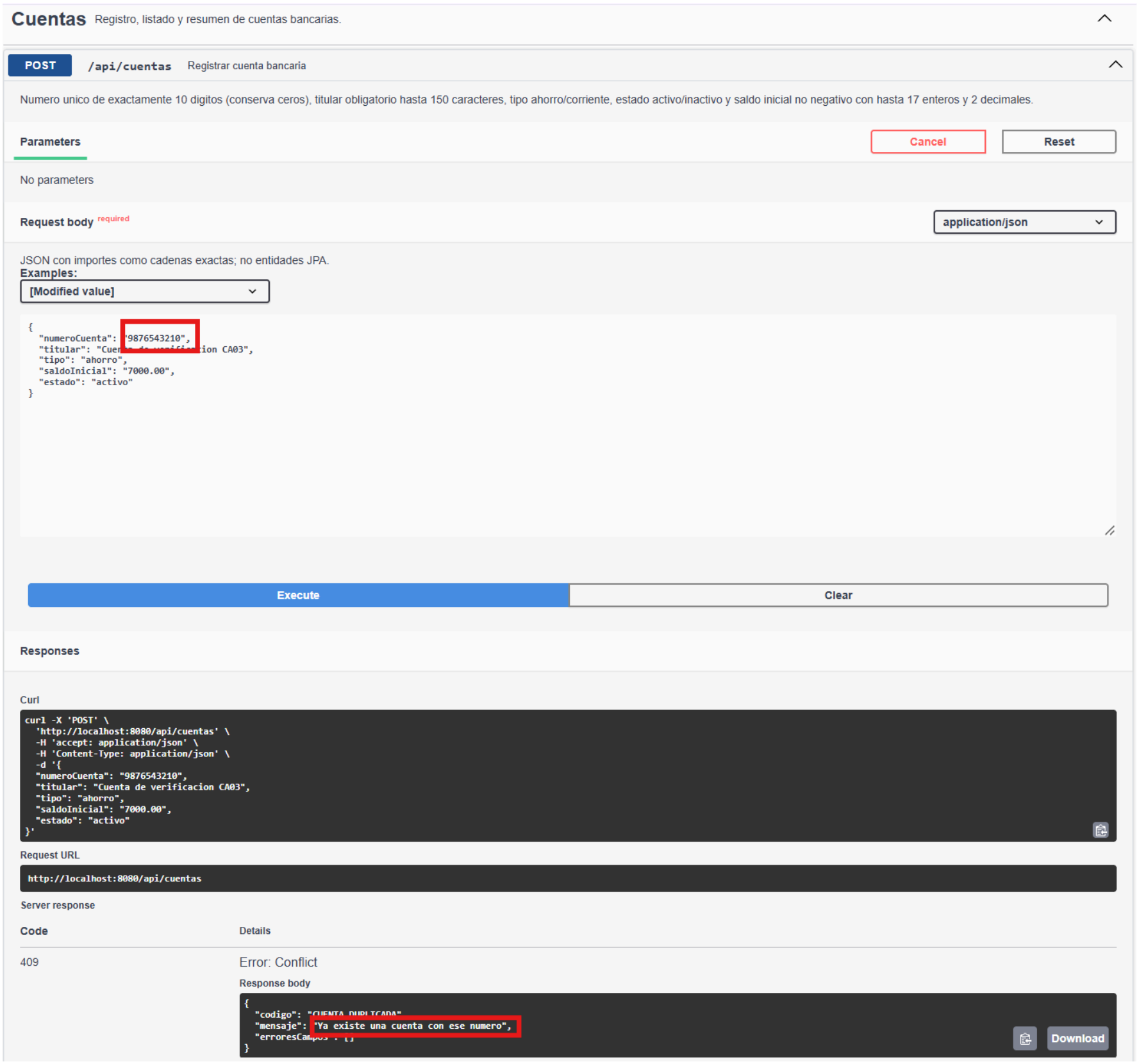
Captura 3. Creacion de cuenta unica



POST /api/cuentas: HTTP 201, ID 5 y número 9876543210; saldo inicial 7000.00.

CA-01: Registro y gestión de cuentas bancarias

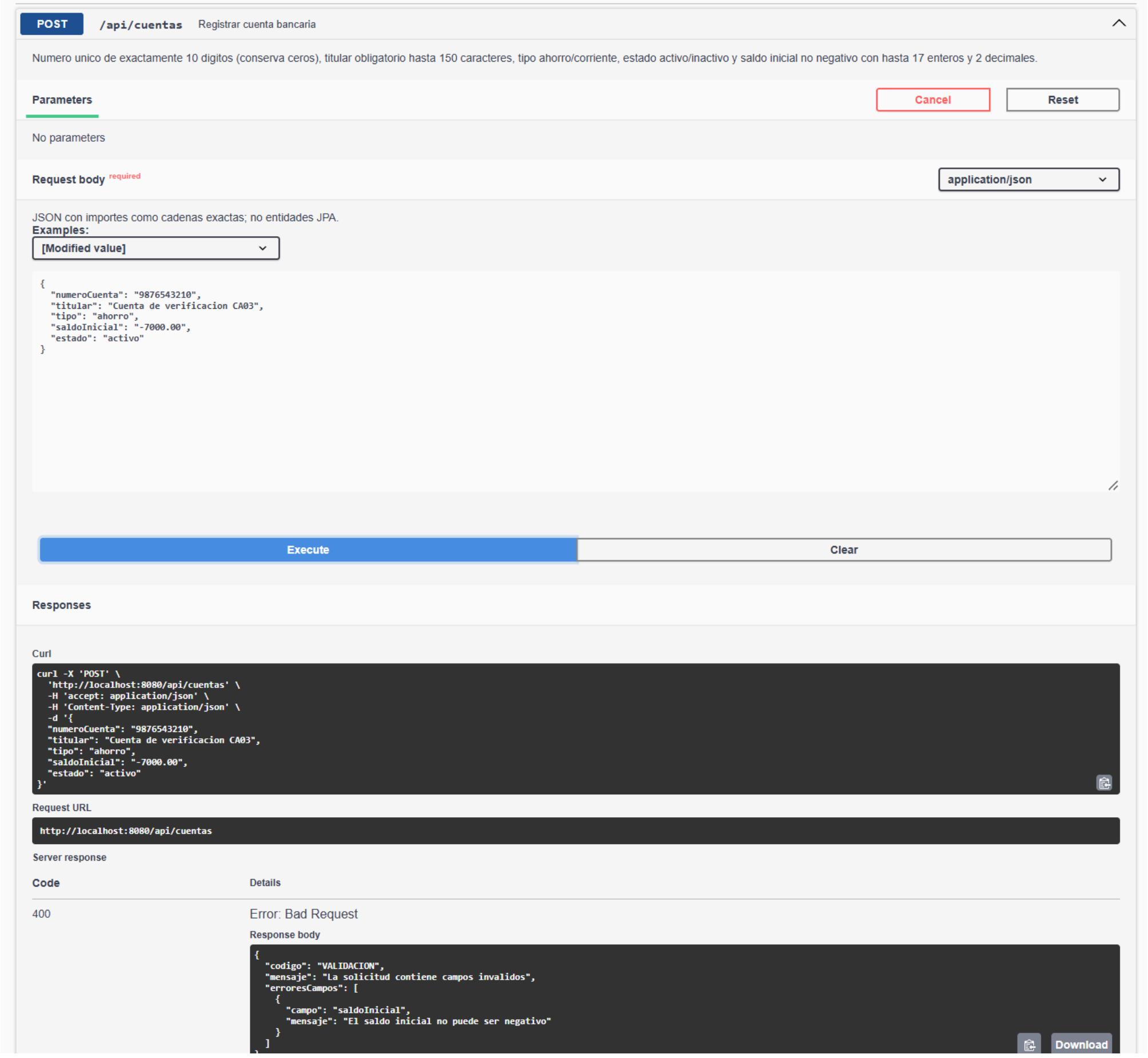
Captura 4. Nr cuenta repetida



Número ya registrado: HTTP 409 y código CUENTA_DUPLICADA.

CA-01: Registro y gestión de cuentas bancarias

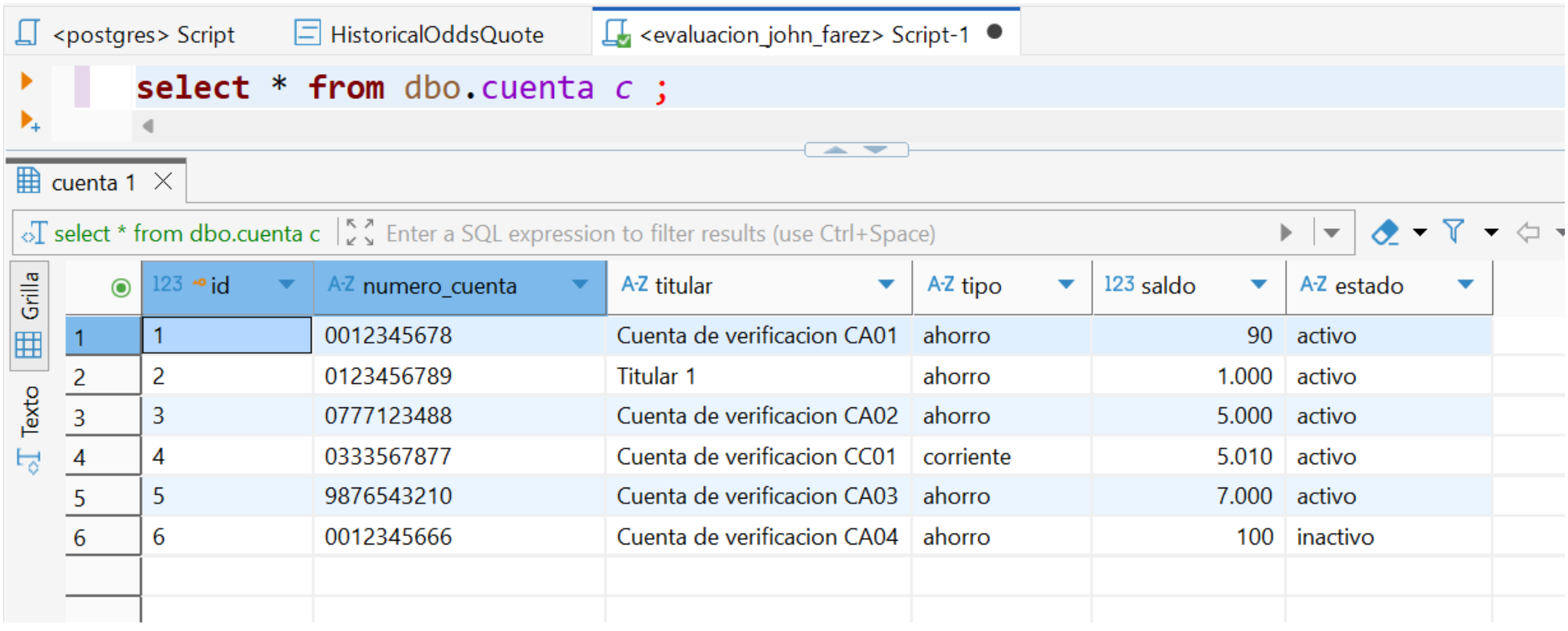
Captura 5. Saldo Negativo



Saldo inicial -7000.00: HTTP 400; mensaje de validación del campo saldoInicial.

CA-01: Registro y gestión de cuentas bancarias

Captura 6. Persistencia de cuentas

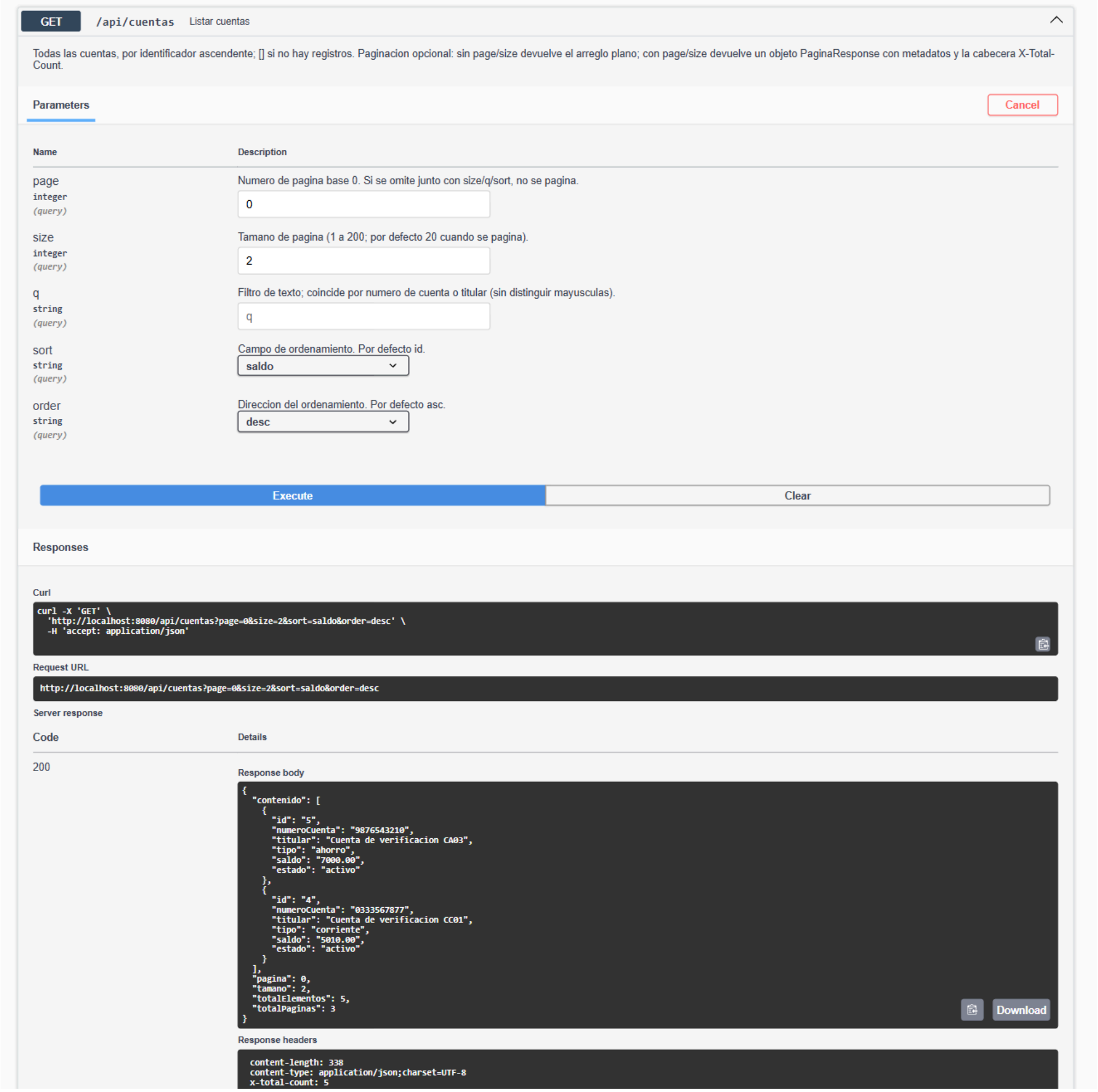


	id	numero_cuenta	titular	tipo	saldo	estado
1	1	0012345678	Cuenta de verificacion CA01	ahorro	90	activo
2	2	0123456789	Titular 1	ahorro	1.000	activo
3	3	0777123488	Cuenta de verificacion CA02	ahorro	5.000	activo
4	4	0333567877	Cuenta de verificacion CC01	corriente	5.010	activo
5	5	9876543210	Cuenta de verificacion CA03	ahorro	7.000	activo
6	6	0012345666	Cuenta de verificacion CA04	ahorro	100	inactivo

Consulta a dbo.cuenta con seis registros persistidos, saldos y estados.

CA-01: Registro y gestión de cuentas bancarias

Captura 7. GET api cuentas



GET paginado: HTTP 200, size=2, sort=saldo, order=desc; metadatos y cuentas devueltas.

CA-02: Transferencias entre cuentas con validaciones de negocio

Requisito del documento oficial

Criterio: El sistema debe exponer un endpoint REST (**POST /api/transferencias**) que ejecute transferencias entre cuentas. Debe validar: que ambas cuentas existan y estén activas, que la cuenta origen tenga saldo suficiente y que el monto sea mayor a cero. La operación debe ser atómica (actualizar saldos de ambas cuentas en una sola transacción de base de datos usando @Transactional). En caso de error, retornar un mensaje descriptivo con el código HTTP apropiado (400, 404, 422).

Respuesta al criterio

POST /api/transferencias retorna HTTP 201 para una transferencia de 10.00. Se observan rechazos 422 por cuenta inactiva y saldo insuficiente, 404 por destino inexistente y 400 por monto negativo. SQL muestra débito de 10.00 y crédito de 10.00; el total de ambas cuentas se conserva en 1090.00. El código capturado contiene @Transactional(rollbackOn = Exception.class), actualización de saldos y registro.

Capturas que acompañan la respuesta

1. Actualización de saldos -registro de transferencias
2. Atomicidad
3. Cuenta inactiva
4. Cuenta inexistente
5. Monto negativo
6. Saldo insuficiente
7. Saldos actualizados
8. Transferencia exitosa
9. Validación de cuentas

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-02: Transferencias entre cuentas con validaciones de negocio

Captura 1. Actualización de saldos -registro de transferencias

```
        origen.saldo = origen.saldo.subtract(monto);
        destino.saldo = destino.saldo.add(monto);

        cuentas.flush();
        var transferencia = mapper.toEntity(request);
        transferencia.monto = monto;
        transferencia.fecha = ctx.ahora();
        transferencia.estado = sospechosa ? "en_revision" : "completada";
        transferencia.claveIdempotencia = ctx.clave();
        transferencias.registrar(transferencia);
        LOG.infof("Transferencia ejecutada: id=%s estado=%s origen=%s destino=%s monto=%s clave=%s",
            transferencia.id, transferencia.estado, origen.id, destino.id, monto.toPlainString(), ctx.clave
            ());
        return new Resultado(mapper.toResponse(transferencia), null);
```

Débito, crédito, flush y registro de transferencia con estado e identificador de idempotencia.

CA-02: Transferencias entre cuentas con validaciones de negocio

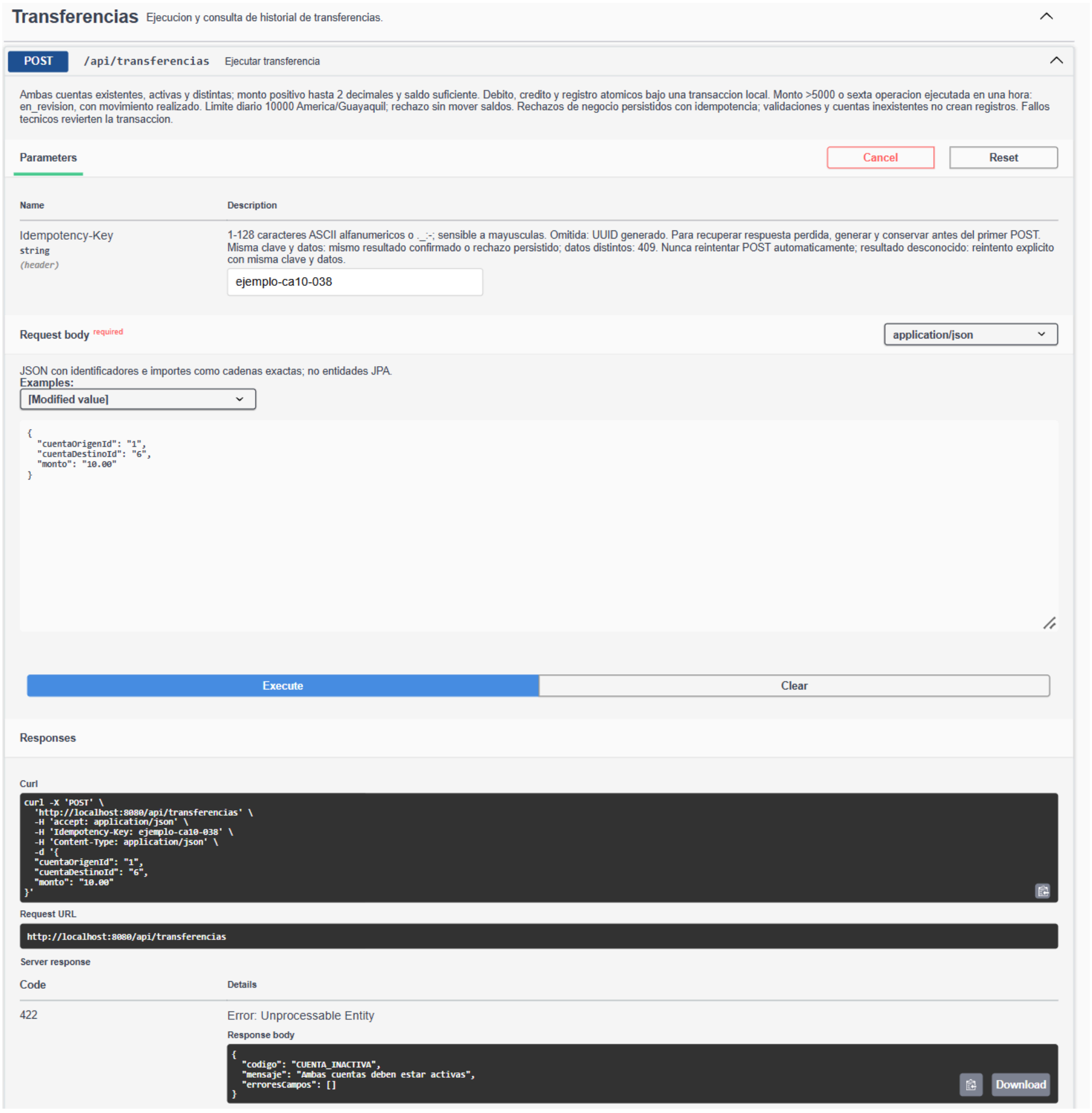
Captura 2. Atomicidad

```
    @Transactional(rollbackOn = Exception.class)
    public Resultado decidir(CrearTransferenciaRequest request, String
    claveSolicitada) {
```

Método decidir anotado con @Transactional(rollbackOn = Exception.class).

CA-02: Transferencias entre cuentas con validaciones de negocio

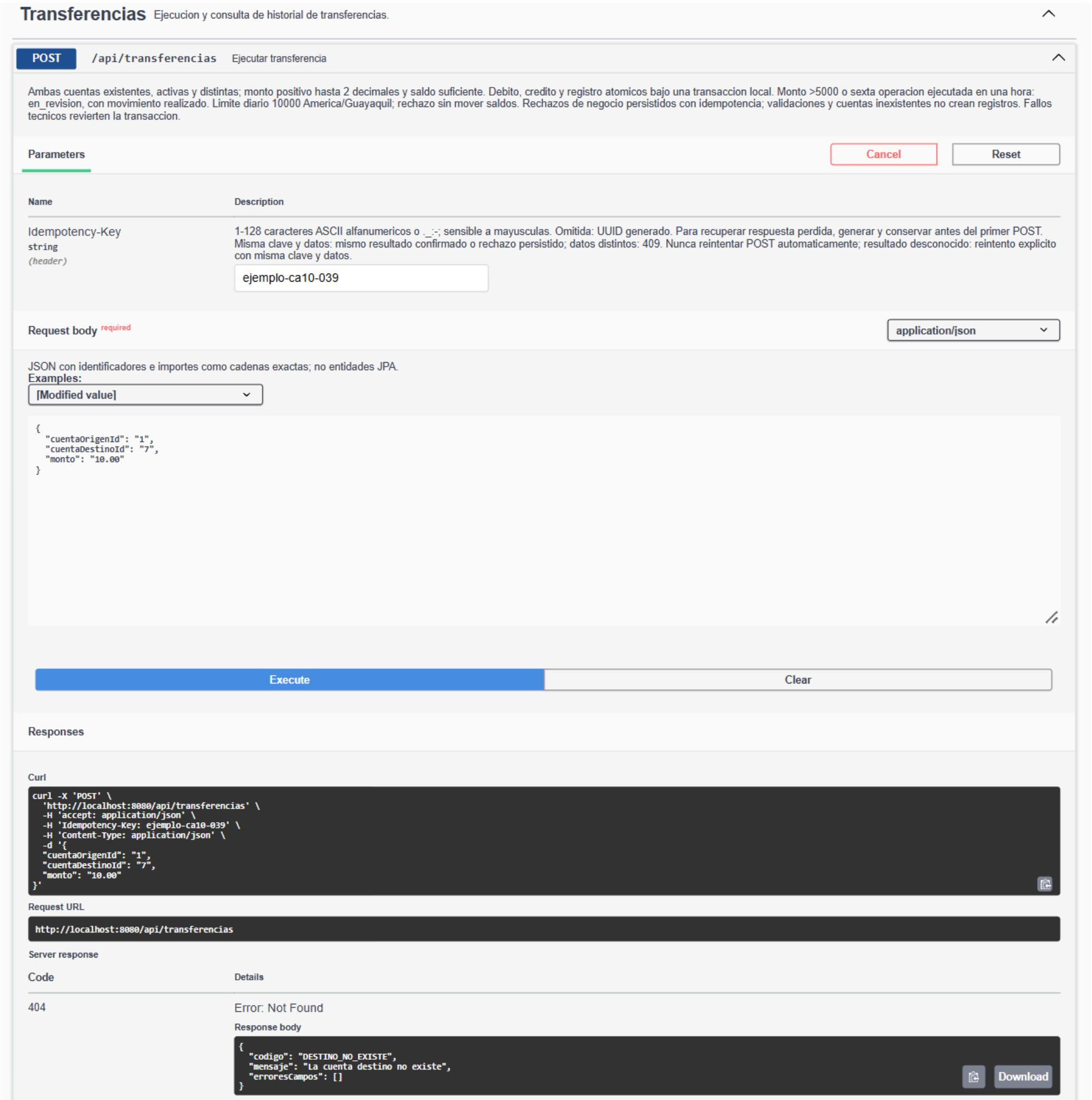
Captura 3. Cuenta inactiva



Destino inactivo: HTTP 422 y código CUENTA_INACTIVA.

CA-02: Transferencias entre cuentas con validaciones de negocio

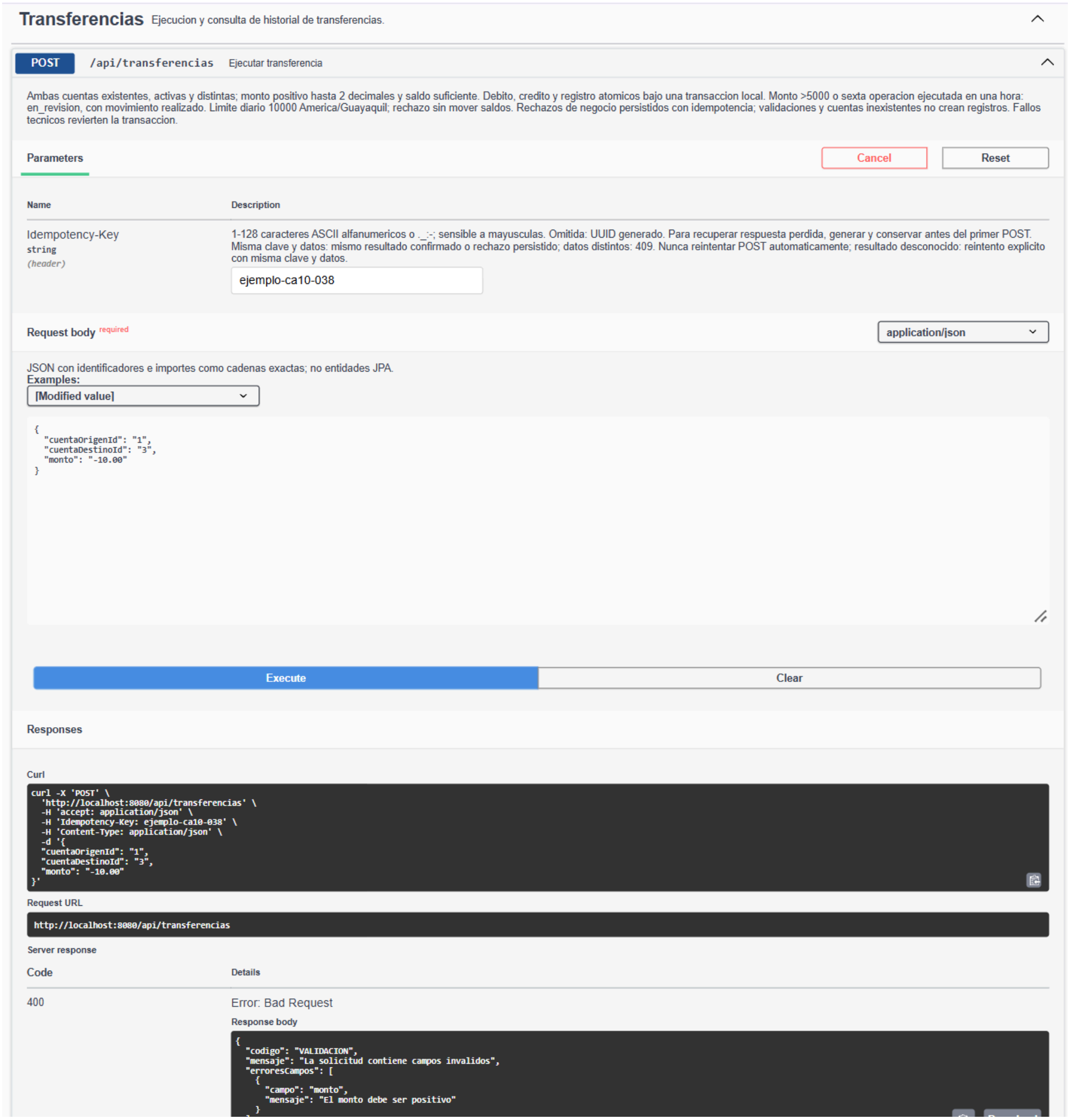
Captura 4. Cuenta inexistente



Destino inexistente: HTTP 404 y código DESTINO_NO_EXISTE.

CA-02: Transferencias entre cuentas con validaciones de negocio

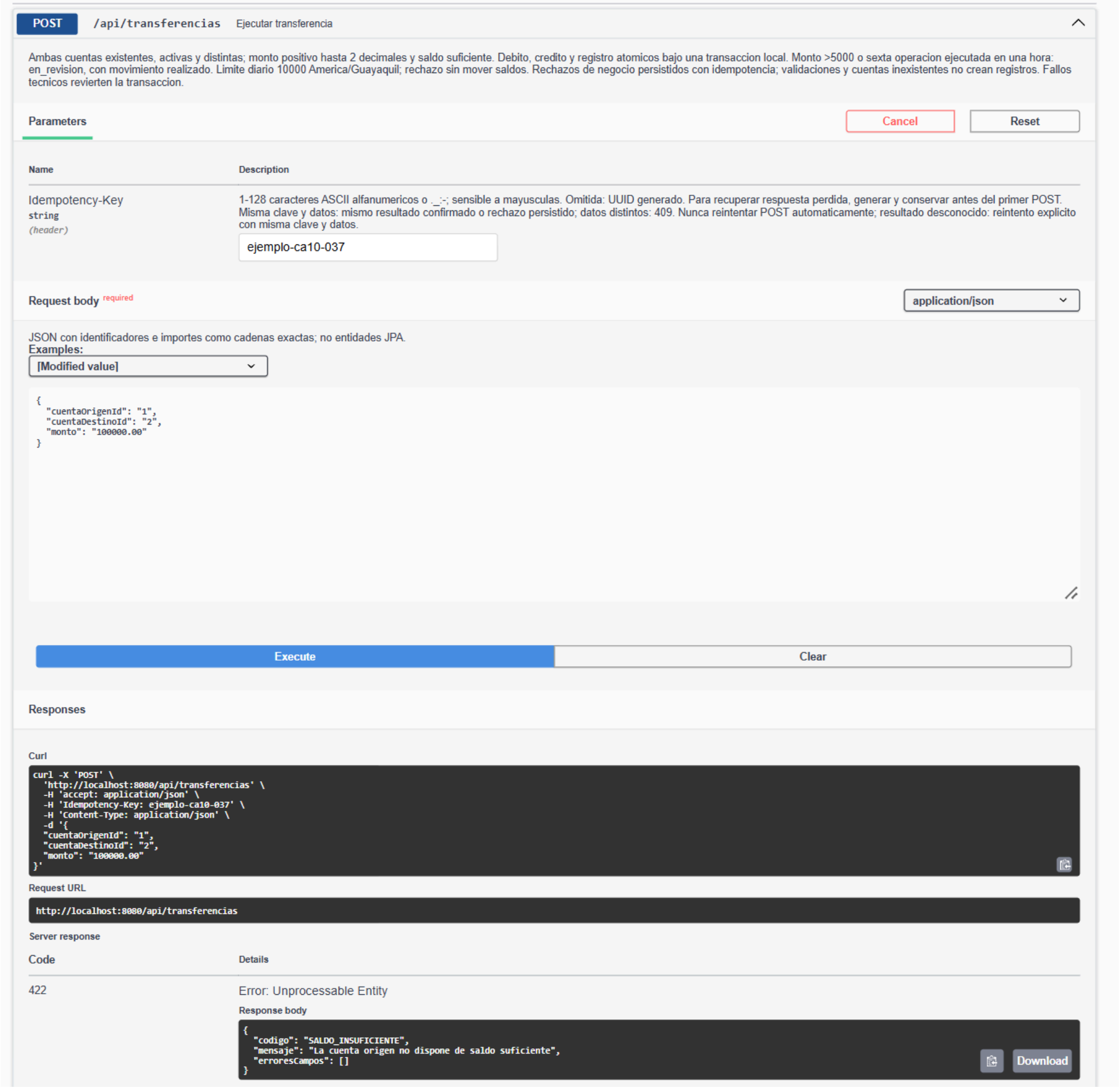
Captura 5. Monto negativo



Monto -10: HTTP 400 y mensaje de monto positivo.

CA-02: Transferencias entre cuentas con validaciones de negocio

Captura 6. Saldo insuficiente



Monto 100000.00: HTTP 422 y código SALDO_INSUFICIENTE.

CA-02: Transferencias entre cuentas con validaciones de negocio

Captura 7. Saldos actualizados

```
select * from dbo.cuenta c where c.id in (1,2);
-- SALDO ANTES
```

id	numero_cuenta	titular	tipo	saldo	estado
1	0012345678	Cuenta de verificacion CA01	ahorro	79,75	activo
2	0123456789	Titular 1	ahorro	1.010,25	activo

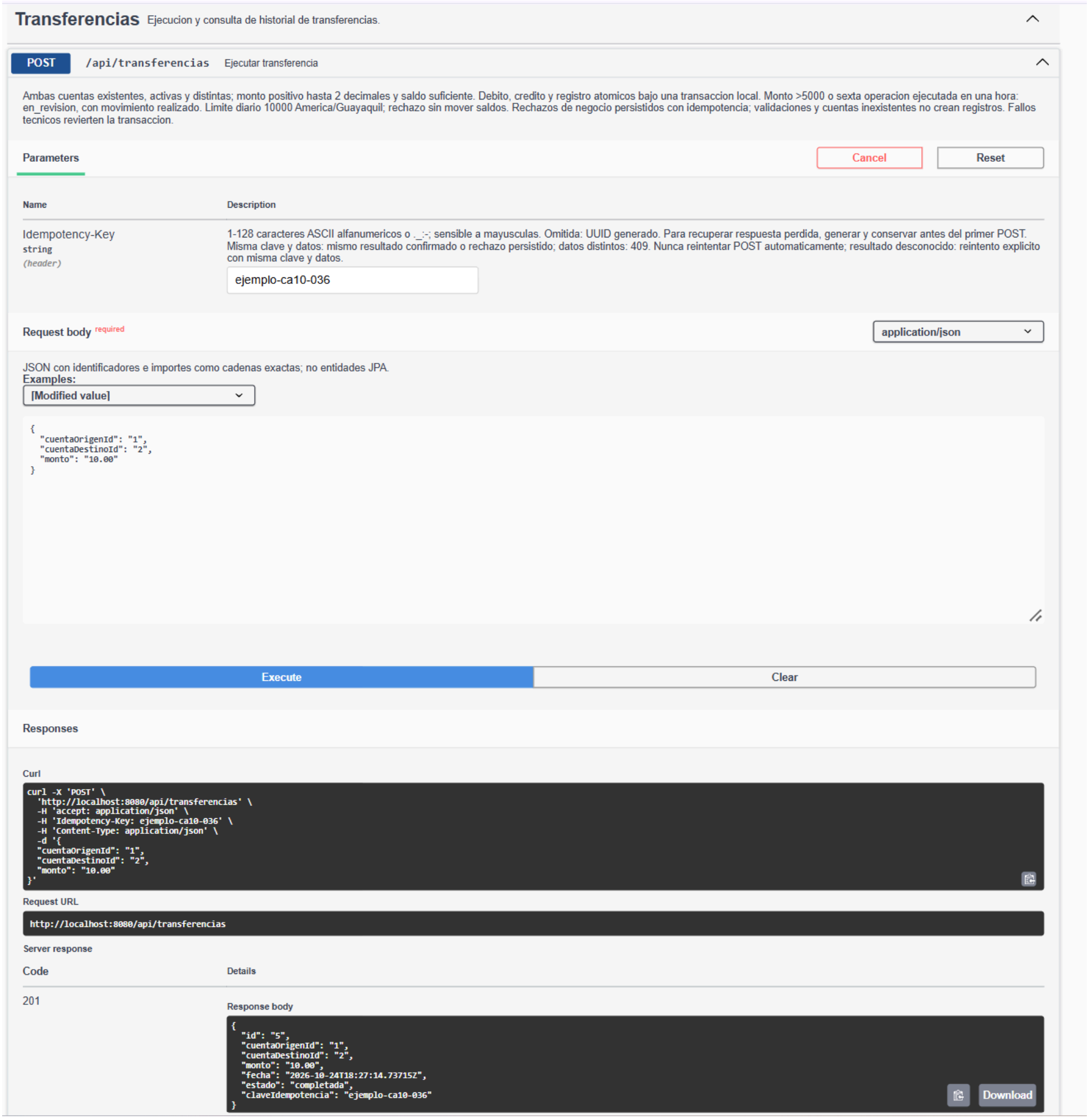
```
select * from dbo.cuenta c where c.id in (1,2);
-- SALDO DESPUES
```

id	numero_cuenta	titular	tipo	saldo	estado
1	0012345678	Cuenta de verificacion CA01	ahorro	69,75	activo
2	0123456789	Titular 1	ahorro	1.020,25	activo

Antes/después: origen 79.75 → 69.75; destino 1010.25 → 1020.25. Total conservado: 1090.00.

CA-02: Transferencias entre cuentas con validaciones de negocio

Captura 8. Transferencia exitosa



Transferencia por 10.00 entre cuentas 1 y 2: HTTP 201, ID 5, estado completada.

CA-02: Transferencias entre cuentas con validaciones de negocio

Captura 9. Validación de cuentas

```
        if (origen == null) {
            throw fallo(404, "ORIGEN_NO_EXISTE", "La cuenta origen no existe");
        }
        if (destino == null) {
            throw fallo(404, "DESTINO_NO_EXISTE", "La cuenta destino no existe");
        }
```

Comprobaciones de origen y destino inexistentes con errores HTTP 404.

CA-03: Detección de operaciones sospechosas

Requisito del documento oficial

Criterio: Implementar un servicio que marque automáticamente una transferencia como "requiere revisión" cuando se cumpla alguna de estas condiciones: (a) el monto individual supera $5,000, o (b) la cuenta origen ha realizado más de 5 transferencias en la última hora. El servicio debe invocarse automáticamente durante el proceso de transferencia y registrar el estado correspondiente en la entidad.

Respuesta al criterio

Una transferencia de 6000.00 aparece persistida con estado en_revision. Otra captura muestra cinco transferencias completadas y una sexta en_revision desde el origen 3; el POST de esta última devuelve HTTP 201, ID 20 y estado en_revision. El código invoca automáticamente la regla con el monto y el contador de operaciones de la última hora.

Capturas que acompañan la respuesta

1. Detección automática de operaciones sospechosas
2. Monto superior a 5k
3. Tabla de transferencias
4. Transferencia sexta en revisión

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-03: Detección de operaciones sospechosas

Captura 1. Detección automática de operaciones sospechosas

```
boolean sospechosa = reglas.sospechosa(monto, transferencias.
ejecutadasEnHora(origen.id, ahora));
origen.saldo = origen.saldo.subtract(monto);
destino.saldo = destino.saldo.add(monto);

cuentas.flush();
var transferencia = mapper.toEntity(request);
transferencia.monto = monto;
transferencia.fecha = ahora;
transferencia.estado = sospechosa ? "en_revision" : "completada";
transferencia.claveIdempotencia = clave;
transferencias.registrar(transferencia);
```

Invocación automática de reglas.sospechosa y contador ejecutadasEnHora; persistencia de en_revision.

CA-03: Detección de operaciones sospechosas

Captura 2. Monto superior a 5k

transferencia 1

SELECT * FROM dbo.transferencia where id =10

Enter a SQL expression to filter results (use Ctrl+Space)

	id	cuenta_origen_id	cuenta_destino_id	monto	fecha	estado	clave_idempotencia
1	10	5	1	6.000	2026-10-24 18:52:59.976 +0000	en_revision	ejemplo-ca10-042

Transferencia ID 10, monto 6000.00 y estado en_revision en la tabla de transferencias.

CA-03: Detección de operaciones sospechosas

Captura 3. Tabla de transferencias

```
SELECT id, cuenta_origen_id, monto, estado
FROM dbo.transferencia
WHERE cuenta_origen_id = 3
ORDER BY fecha DESC;
```

ferencia 1

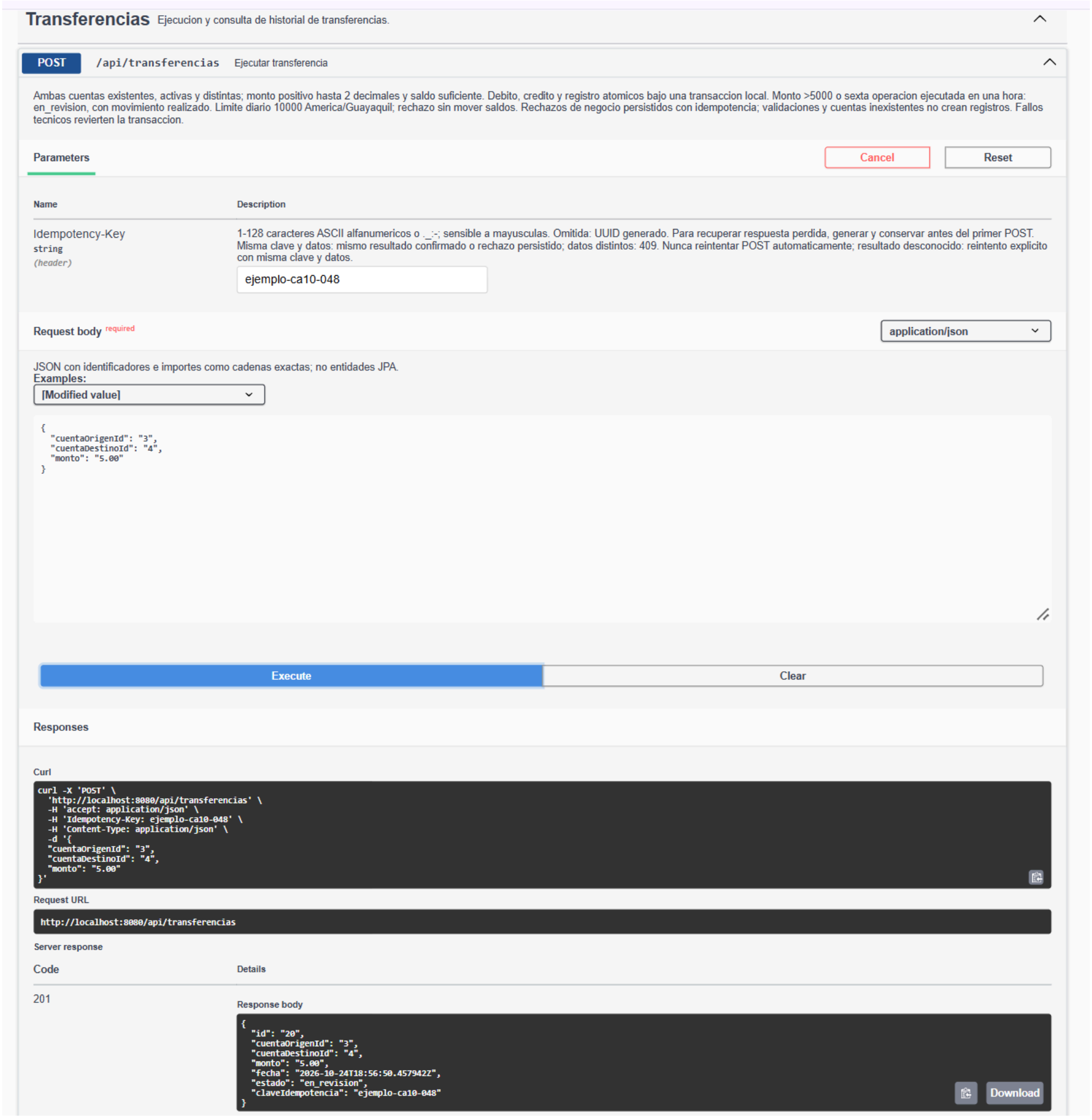
CT id, cuenta_origen_id, monto, estado FROM dbo.transferei | Enter a SQL expression to filter results (use Ctrl

id	cuenta_origen_id	monto	estado
20	3	5	en_revision
19	3	5	completada
18	3	5	completada
17	3	5	completada
16	3	5	completada
15	3	5	completada

Origen 3: cinco movimientos completada y el ID 20 en_revision, todos por 5.00.

CA-03: Detección de operaciones sospechosas

Captura 4. Transferencia sexta en revisión



Sexta operación del escenario: HTTP 201, ID 20, monto 5.00 y estado en_revision.

CA-04: Control de límite diario de transferencias

Requisito del documento oficial

Criterio: Implementar un servicio que controle que ninguna cuenta pueda transferir más de $10,000 acumulados en un día calendario. Si una transferencia haría que el acumulado del día supere ese límite, debe ser rechazada con un mensaje informativo que indique el monto restante disponible.

Respuesta al criterio

La consulta SQL muestra un acumulado de 10000 para la cuenta origen 7 durante el día local, incluyendo operaciones completada y en_revision. Un nuevo intento por 1000.00 devuelve HTTP 422, código LIMITE_DIARIO_EXCEDIDO y montoRestanteDisponible de 0.00. La implementación capturada calcula el disponible y rechaza importes que lo superan.

Capturas que acompañan la respuesta

1. Acumulado diario de 10000
2. Rechazo por limite diario excedido
3. Validacion del limite diario Java

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-04: Control de límite diario de transferencias

Captura 1. Acumulado diario de 10000

```sql
SELECT cuenta_origen_id,
       SUM(monto) AS total_transferido
FROM dbo.transferencia
WHERE cuenta_origen_id = 7
  AND estado IN ('completada', 'en_revision')
  AND CAST(fecha AT TIME ZONE 'SA Pacific Standard Time' AS DATE)
      = CAST(SYSDATETIMEOFFSET() AT TIME ZONE 'SA Pacific Standard Time' AS DATE)
GROUP BY cuenta_origen_id;
```

ltados 1

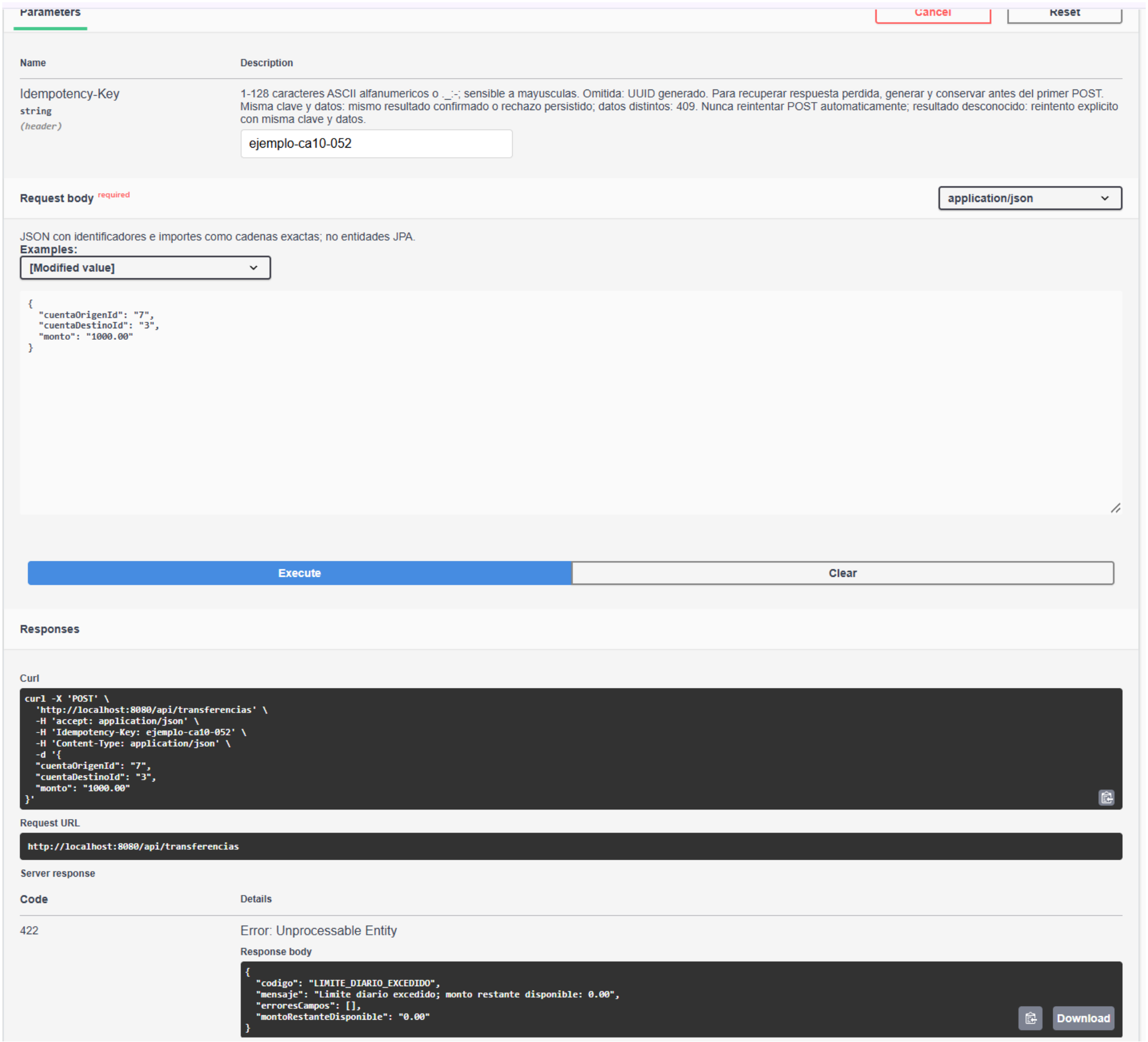
CT cuenta_origen_id, SUM(monto) AS total_transferido FROM | Enter a SQL expression to filter results (use Ctrl+Space)

123 cuenta_origen_id	123 total_transferido
7	10.000

Acumulado de 10000 para origen 7; consulta por día local y estados ejecutados.

CA-04: Control de límite diario de transferencias

Captura 2. Rechazo por limite diario excedido



Intento por 1000.00: HTTP 422 LIMITE_DIARIO_EXCEDIDO; disponible 0.00.

CA-04: Control de límite diario de transferencias

Captura 3. Validacion del limite diario Java

```java
public void validarLimite(BigDecimal acumulado, BigDecimal monto) {
    var disponible = LIMITE.subtract(acumulado).max(new BigDecimal("0.00"));
    if (monto.compareTo(disponible) > 0) {
        throw new OperacionFinancieraException(422, "LIMITE_DIARIO_EXCEDIDO",
            "Limite diario excedido; monto restante disponible: " + disponible.
            toPlainString(), disponible);
    }
}
```

validarLimite calcula disponible con BigDecimal y lo incorpora al error 422.

CA-05: Consulta de historial y resumen de cuenta

Requisito del documento oficial

Criterio: El sistema debe exponer: (a) un endpoint **GET /api/transferencias** que permita consultar el historial de transferencias con filtros opcionales por cuenta origen, cuenta destino, rango de fechas y estado (completada/rechazada/en revisión), retornando resultados ordenados por fecha descendente; y (b) un endpoint **GET /api/cuentas/{id}/resumen** que retorne un resumen con: saldo actual, total transferido hoy, cantidad de transferencias del día y cantidad marcadas como sospechosas.

Respuesta al criterio

Se observan respuestas HTTP 200 del historial general, del filtro combinado por origen 3, destino 4 y estado completada, y del rango de fechas del 24 de octubre de 2026 con offset -05:00. Los registros visibles están ordenados por fecha descendente. El resumen de la cuenta 3 devuelve saldoActual 14970.00, totalTransferidoHoy 30.00, cantidadTransferenciasHoy 6 y cantidadTransferenciasSospechosas 1.

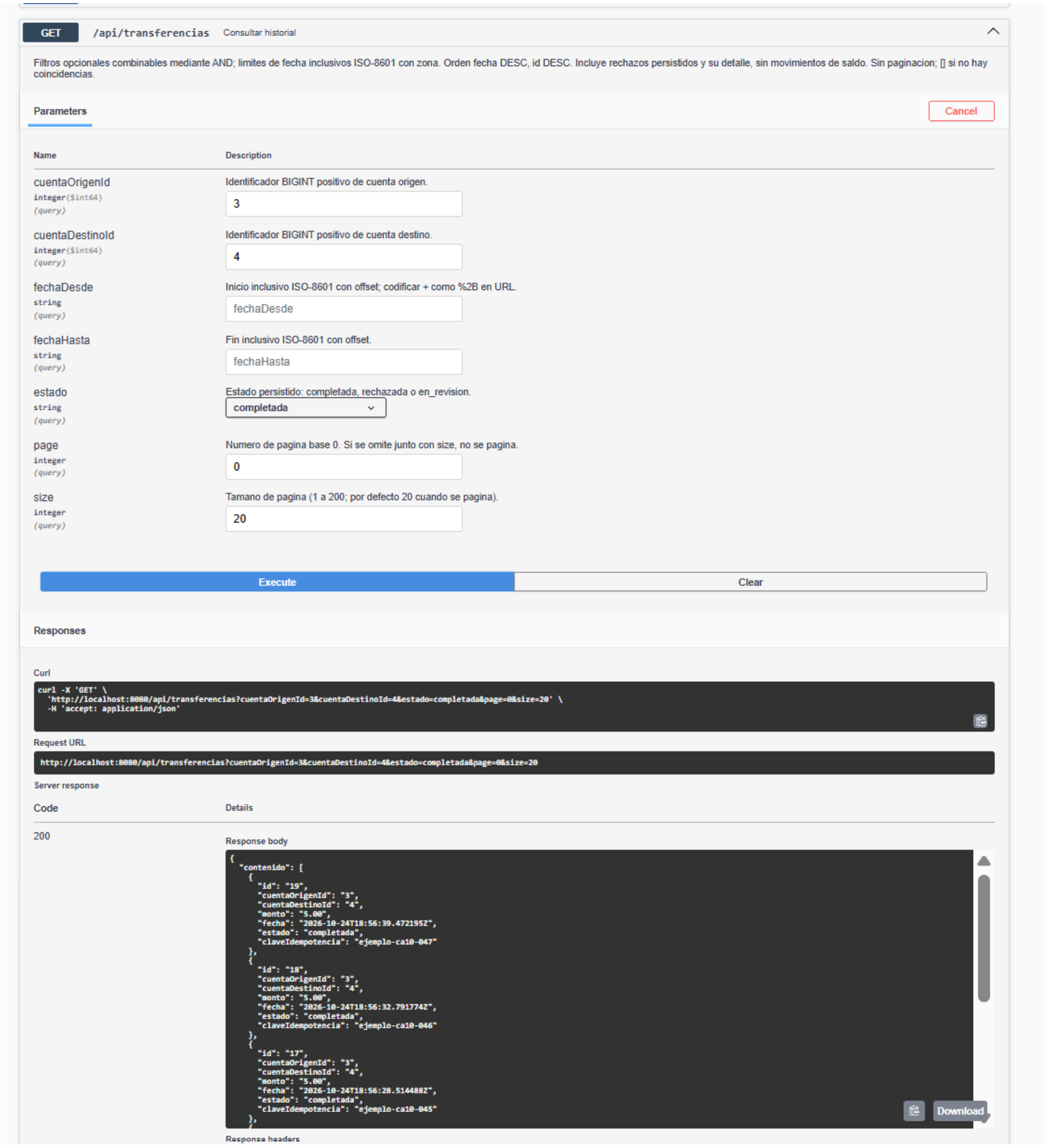
Capturas que acompañan la respuesta

1. Filtros combinados del historial
2. Historial completo de transferencias
3. Historial filtrado por fechas
4. Resumen financiero de cuenta

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-05: Consulta de historial y resumen de cuenta

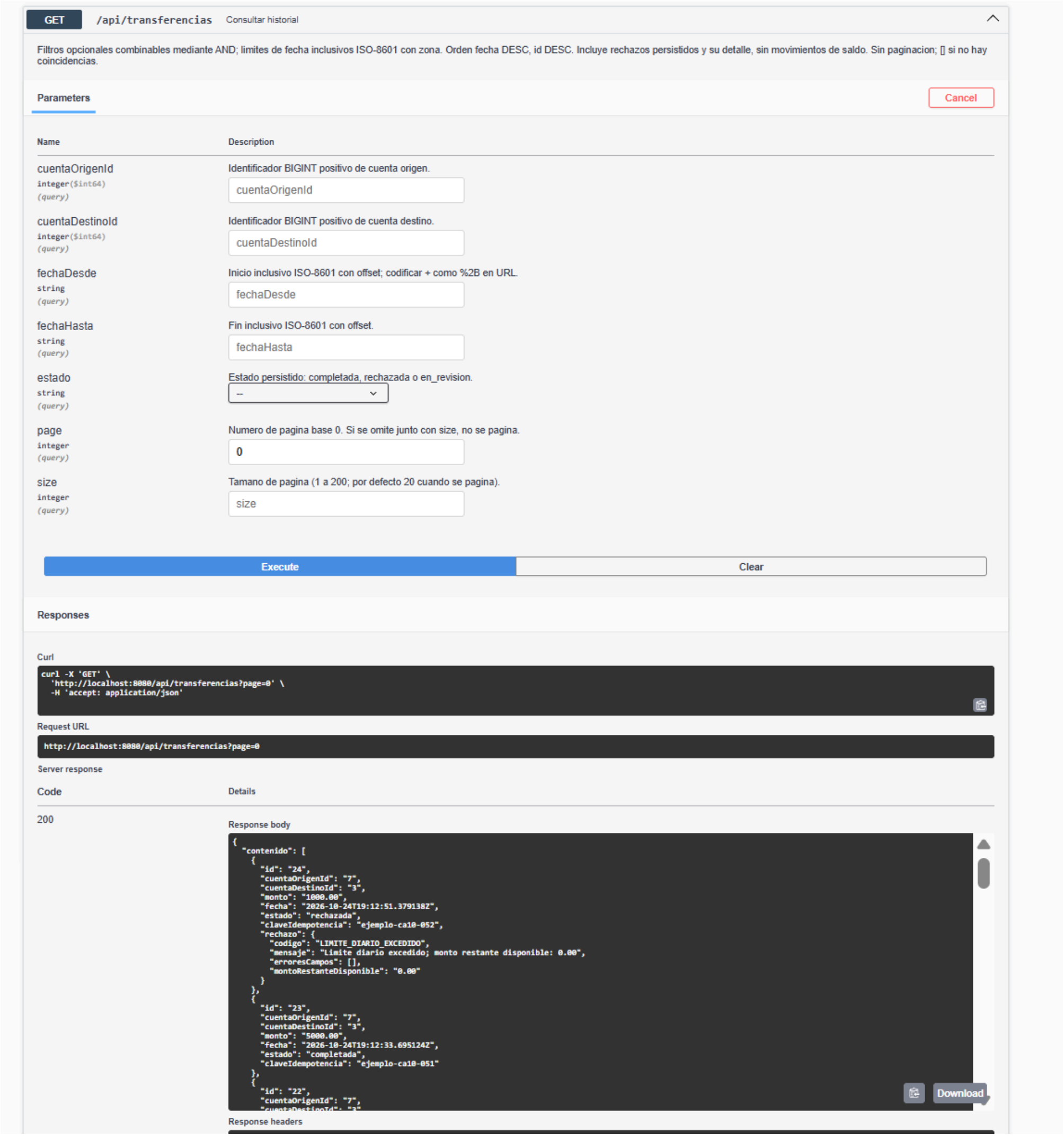
Captura 1. Filtros combinados del historial



Filtro combinado origen=3, destino=4, estado=completada: HTTP 200; fechas visibles descendentes.

CA-05: Consulta de historial y resumen de cuenta

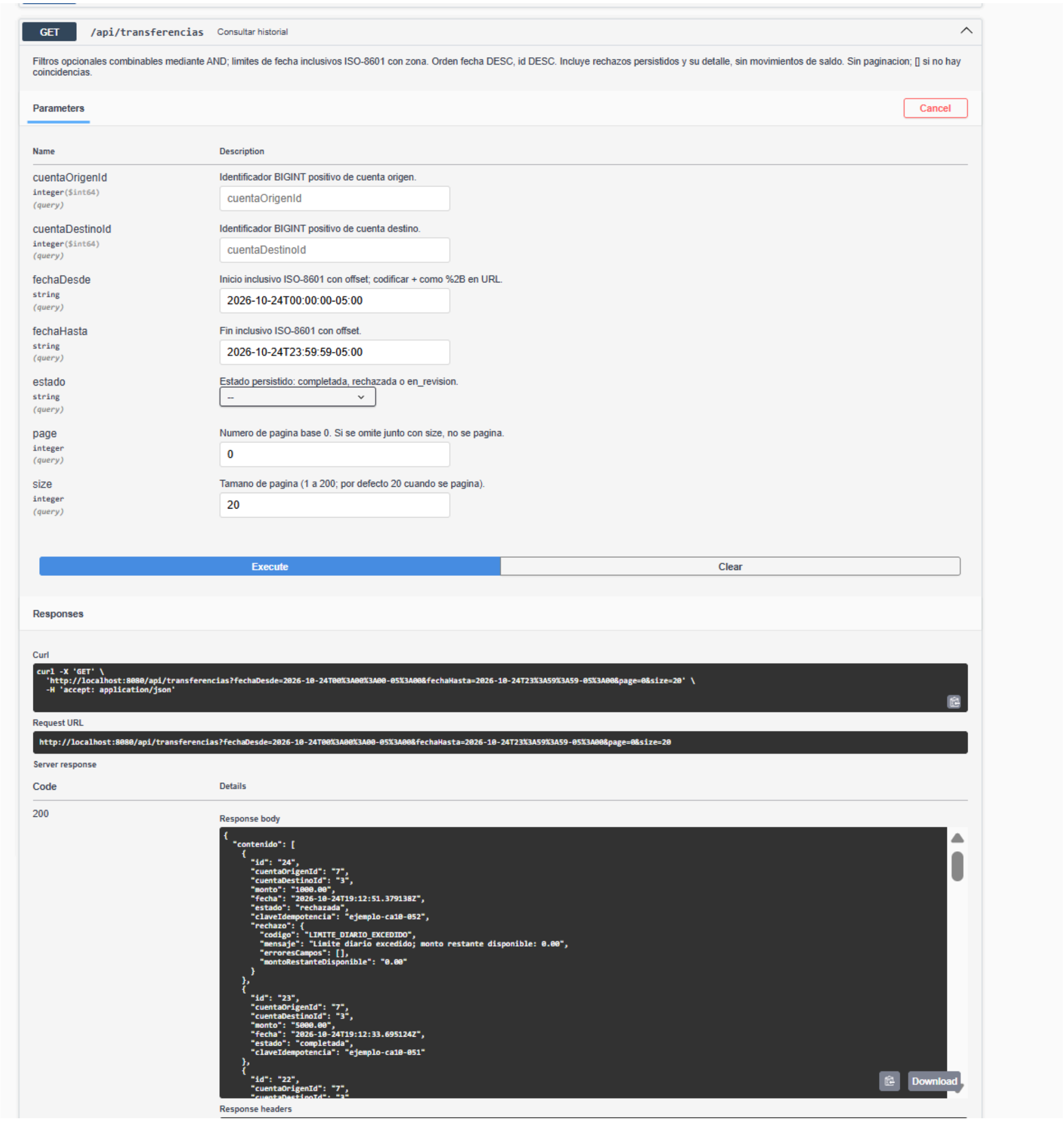
Captura 2. Historial completo de transferencias



Historial sin filtros de negocio, con page=0: HTTP 200; se observa rechazo persistido con su detalle.

CA-05: Consulta de historial y resumen de cuenta

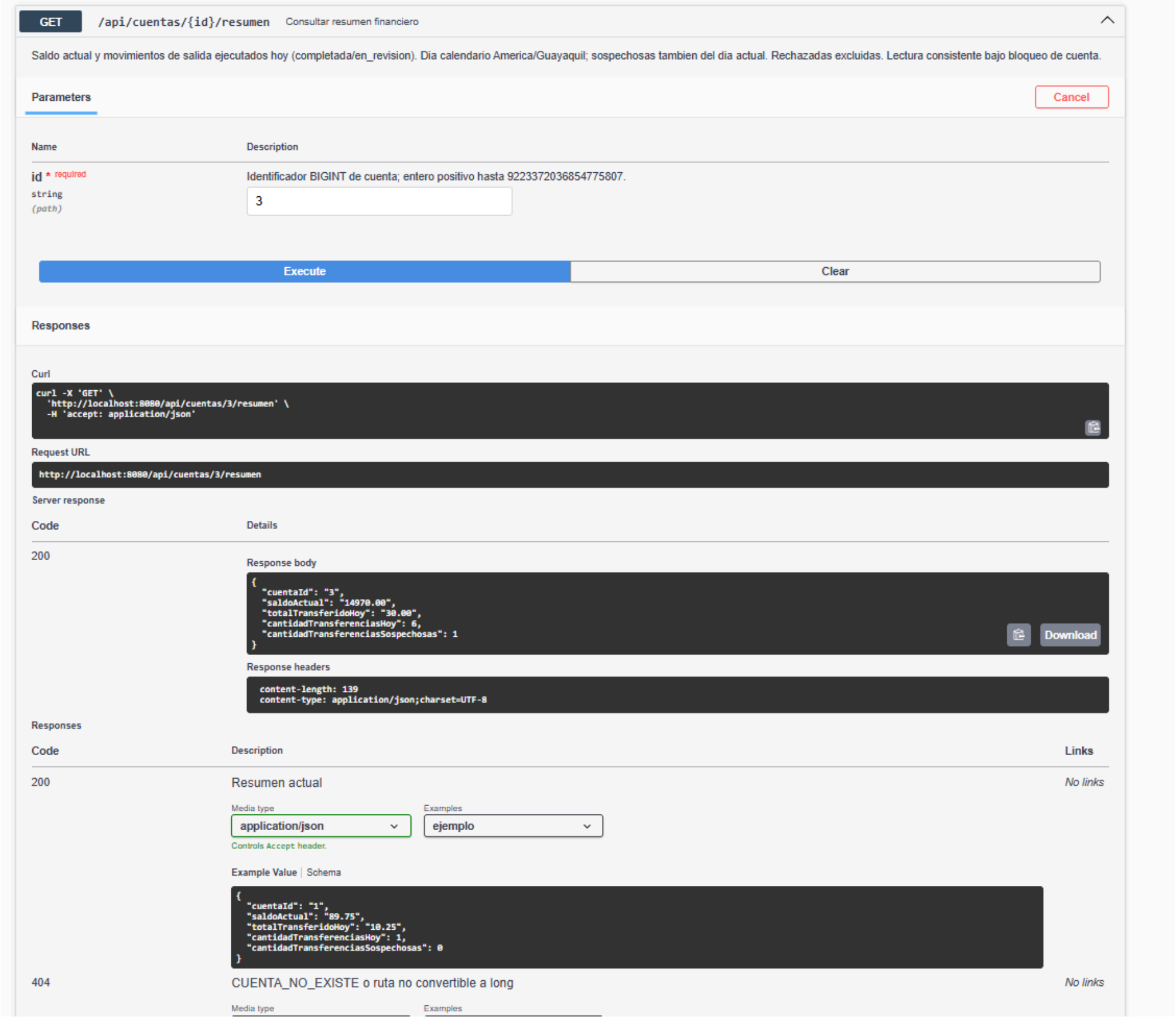
Captura 3. Historial filtrado por fechas



Rango 2026-10-24T00:00:00-05:00 a 2026-10-24T23:59:59-05:00: HTTP 200.

CA-05: Consulta de historial y resumen de cuenta

Captura 4. Resumen financiero de cuenta



Resumen real de cuenta 3: saldo 14970.00, total del día 30.00, seis operaciones y una sospechosa. El ejemplo inferior de Swagger es ilustrativo.

CA-06: Dashboard de cuentas bancarias

Requisito del documento oficial

Criterio: Desarrollar un componente Angular que presente una tabla con las cuentas bancarias registradas, mostrando: número de cuenta, titular, tipo de cuenta, saldo actual y estado. La tabla debe permitir ordenamiento por saldo (ascendente/descendente) y consumir el endpoint **GET /api/cuentas** del backend.

Respuesta al criterio

La tabla Angular muestra siete cuentas con número, titular, tipo, saldo y estado. DevTools registra GET /api/cuentas?page=0&size=10&sort=saldo&order=asc con HTTP 200 y X-Total-Count: 7. Los saldos visibles ascienden desde 100.00 hasta 40000.00, respaldando el consumo real y el orden ascendente.

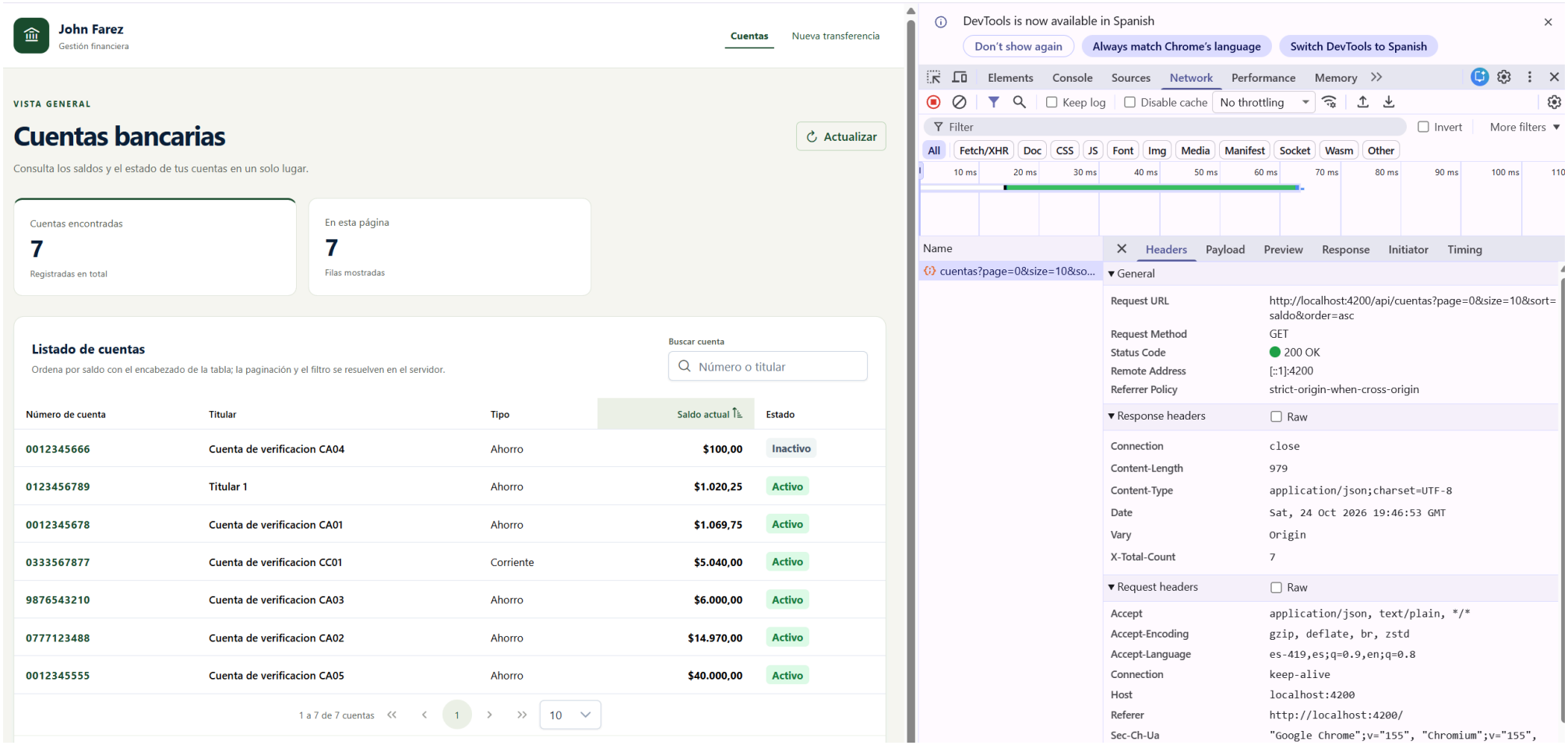
Capturas que acompañan la respuesta

1. Dashboard de cuentas bancarias

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-06: Dashboard de cuentas bancarias

Captura 1. Dashboard de cuentas bancarias



Siete cuentas en orden ascendente de saldo; GET real con order=asc y HTTP 200.

CA-07: Formulario de transferencia

Requisito del documento oficial

Criterio: Implementar un formulario Angular para ejecutar nuevas transferencias. Debe incluir: selección de cuenta origen, selección de cuenta destino, monto y un botón de envío. El formulario debe validar en el cliente que: todos los campos sean requeridos, el monto sea numérico y mayor a cero, y la cuenta origen sea diferente a la destino. Debe mostrar mensajes de éxito o error provenientes del backend.

Respuesta al criterio

El formulario dispone de origen, destino, monto y botón de envío. Se observan mensajes de campos requeridos, rechazo de cuentas iguales y rechazo de monto negativo. La interfaz muestra un error de saldo insuficiente y confirmaciones con identificador, importe y cuentas. DevTools acredita POST /api/transferencias con HTTP 201 para la transferencia ID 27.

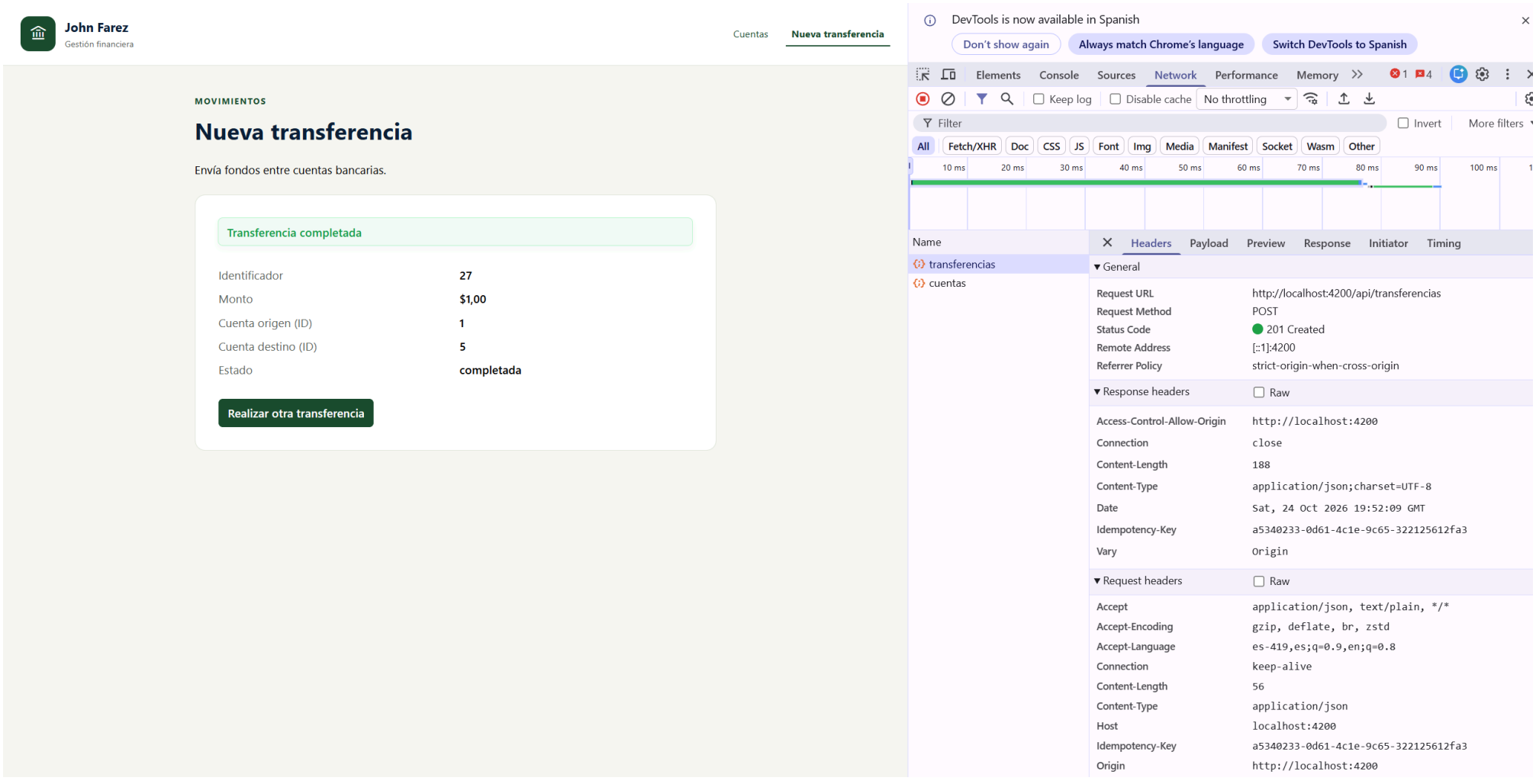
Capturas que acompañan la respuesta

1. Consumo POST transferencias Angular
2. Error de saldo insuficiente Angular
3. Transferencia exitosa Angular
4. Validacion de campos obligatorios
5. Validacion de cuentas iguales
6. Validacion de monto negativo

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-07: Formulario de transferencia

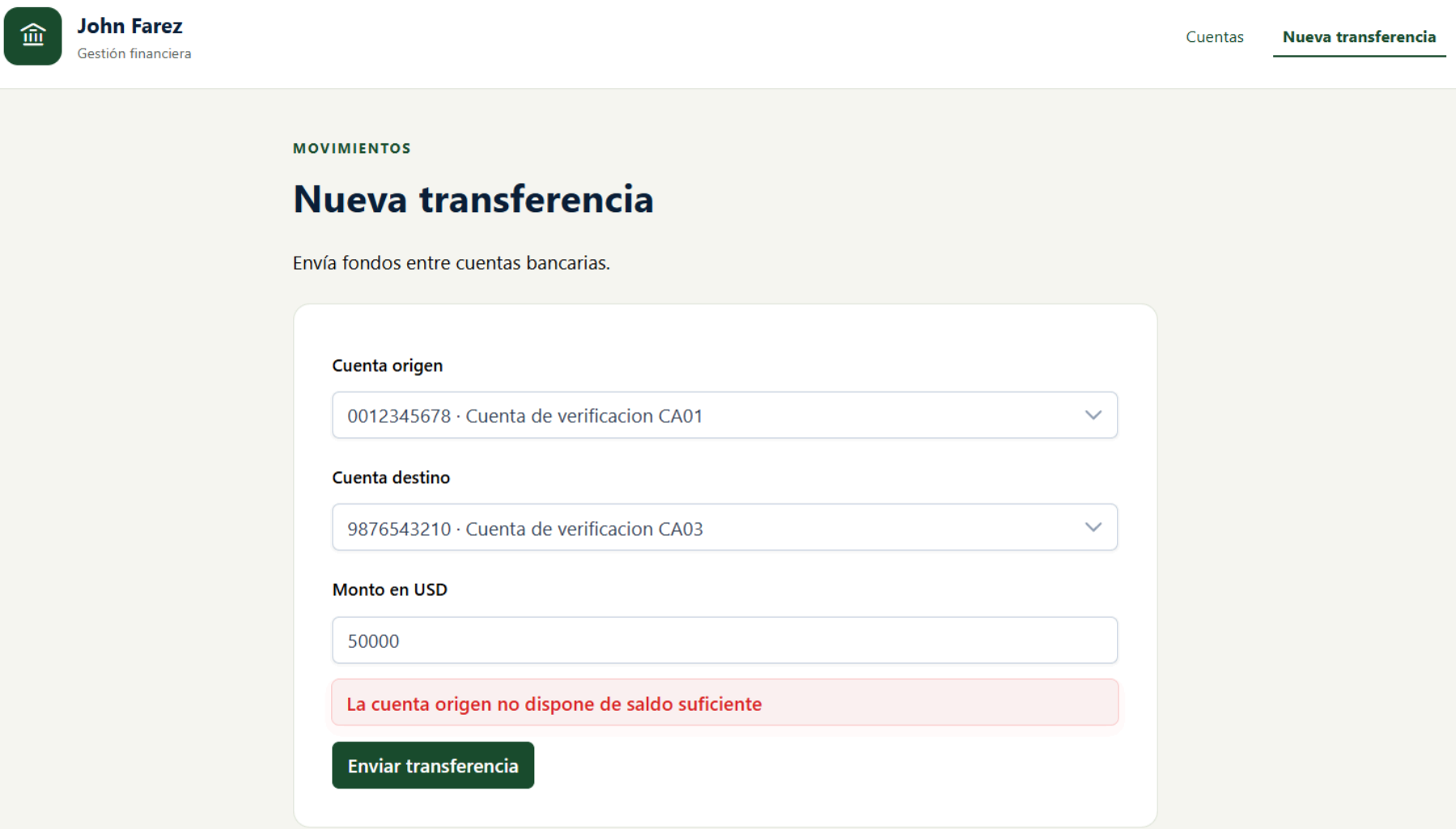
Captura 1. Consumo POST transferencias Angular



Confirmación ID 27 por 1.00 y POST /api/transferencias HTTP 201. Se observa cabecera Idempotency-Key.

CA-07: Formulario de transferencia

Captura 2. Error de saldo insuficiente Angular



Mensaje: La cuenta origen no dispone de saldo suficiente, para un intento de 50000.

CA-07: Formulario de transferencia

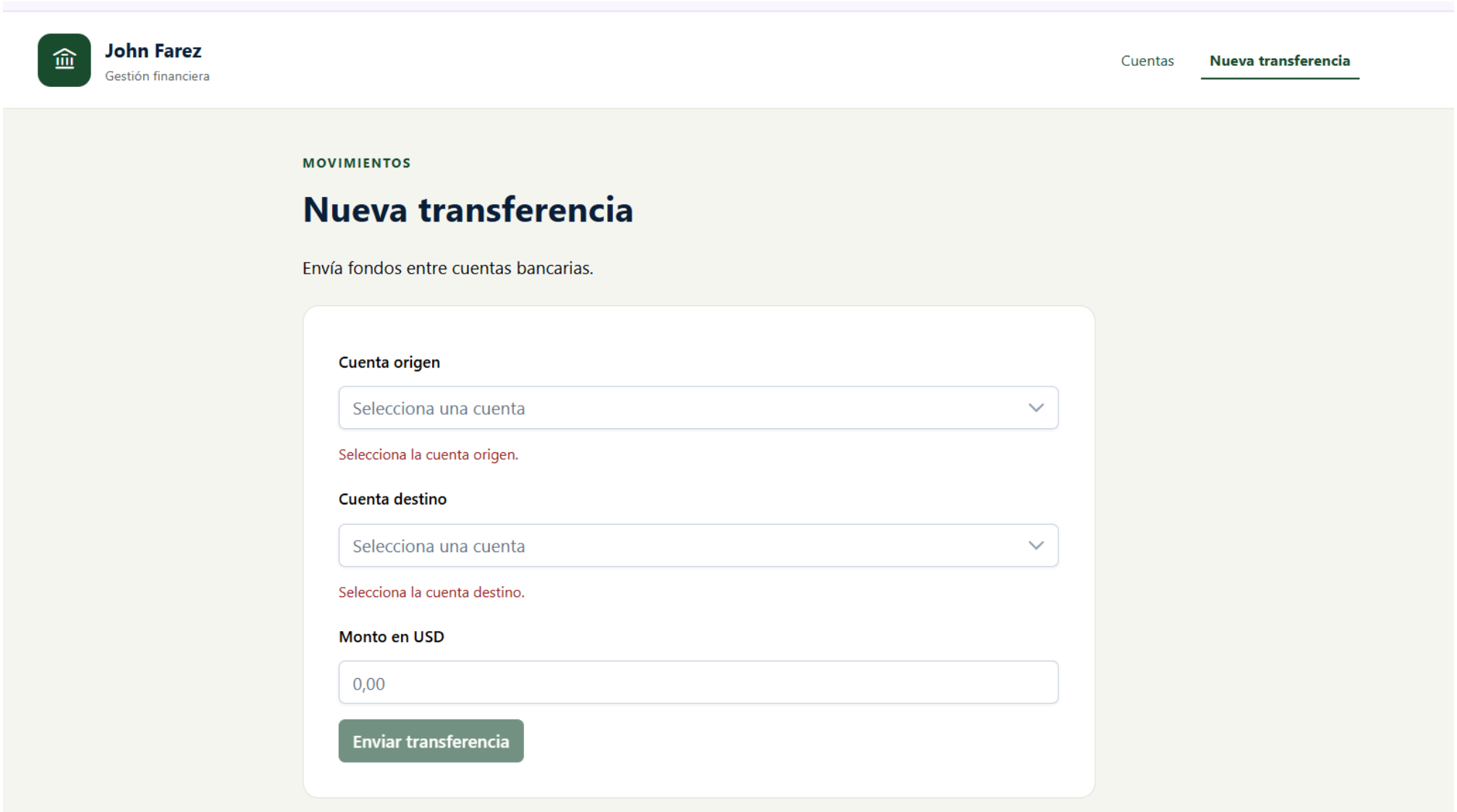
Captura 3. Transferencia exitosa Angular



Confirmación ID 25 por 5.00, origen 1, destino 5 y estado completada.

CA-07: Formulario de transferencia

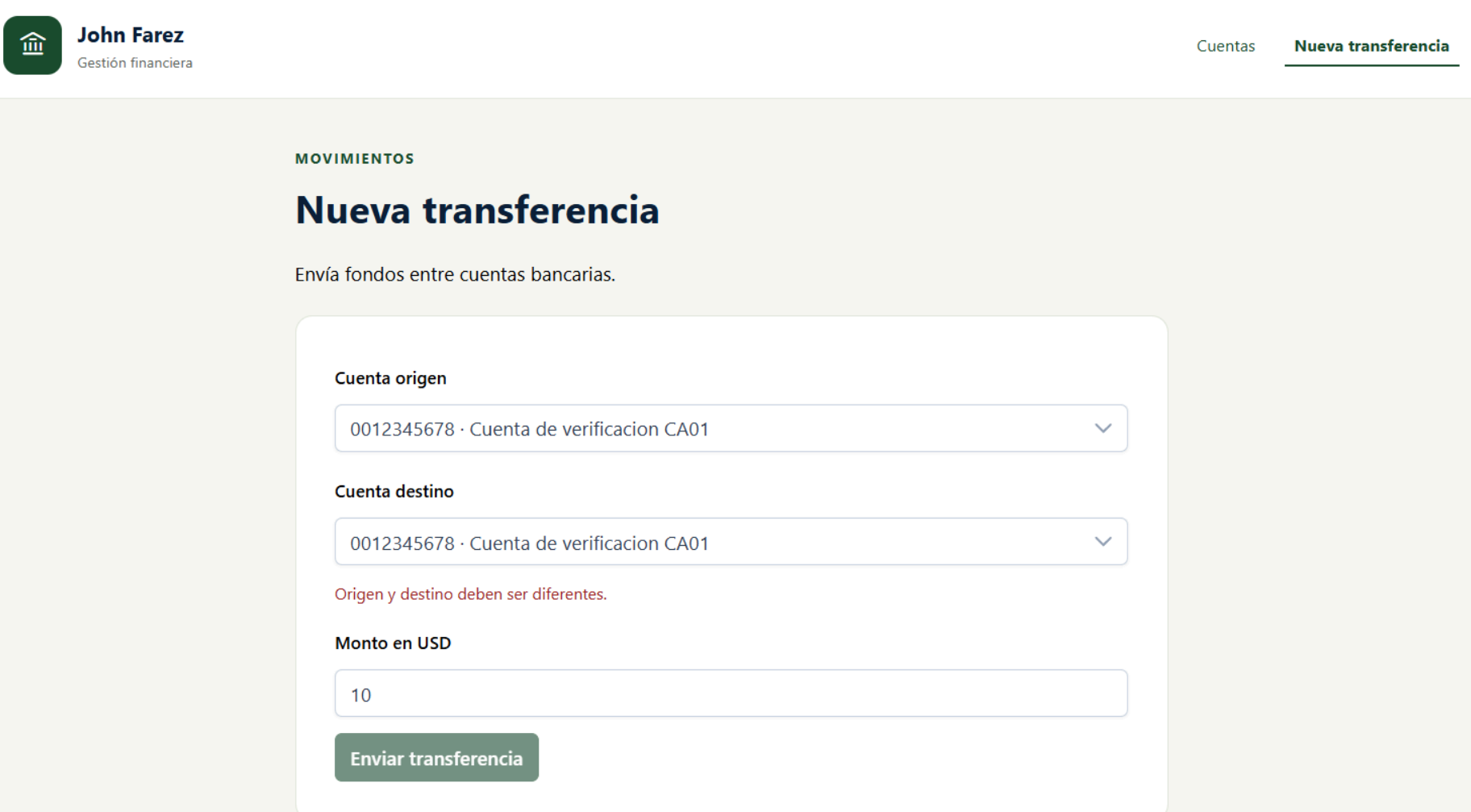
Captura 4. Validacion de campos obligatorios



Mensajes de selección obligatoria de origen/destino y botón de envío deshabilitado.

CA-07: Formulario de transferencia

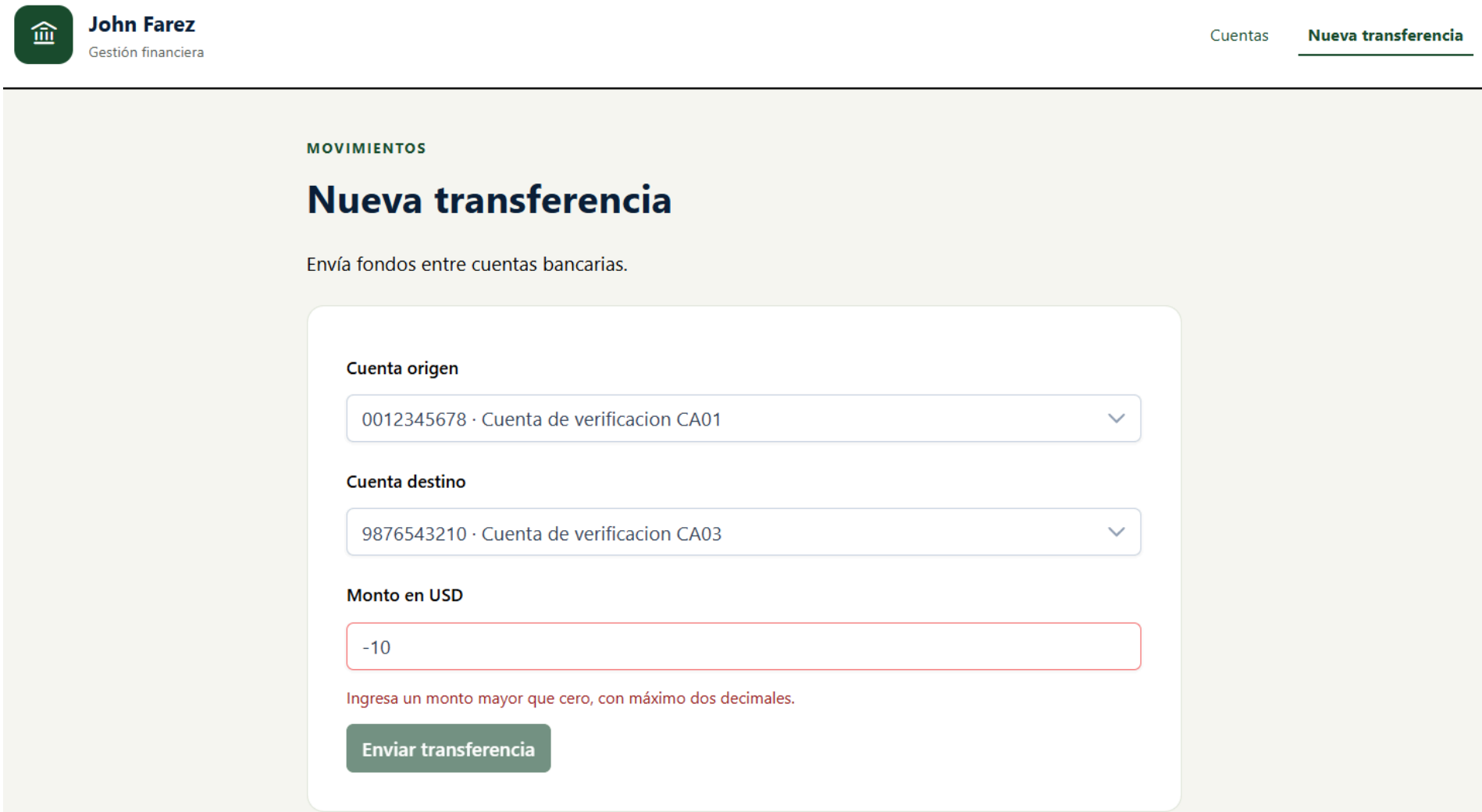
Captura 5. Validacion de cuentas iguales



Origen y destino iguales: mensaje de validación y botón deshabilitado.

CA-07: Formulario de transferencia

Captura 6. Validacion de monto negativo



Monto -10: mensaje que exige un valor mayor que cero con máximo dos decimales; envío deshabilitado.

CA-08: Arquitectura y separación de responsabilidades

Requisito del documento oficial

Criterio: La arquitectura del código fuente debe seguir un enfoque en capas con separación clara de responsabilidades: Resources/Controllers, Services, Repositories. La lógica de negocio no debe residir en controladores ni repositorios. El código debe demostrar modularidad y escalabilidad.

Respuesta al criterio

El árbol del backend único presenta paquetes resource, service, repository, entity, dto, mapper y exception. Se identifican CuentaResource, TransferenciaResource, CuentaRepository, TransferenciaRepository y servicios especializados. Las capturas de CA-02 a CA-04 sitúan la operación transaccional y las reglas financieras en servicios. ApiExceptionMapper centraliza respuestas de error.

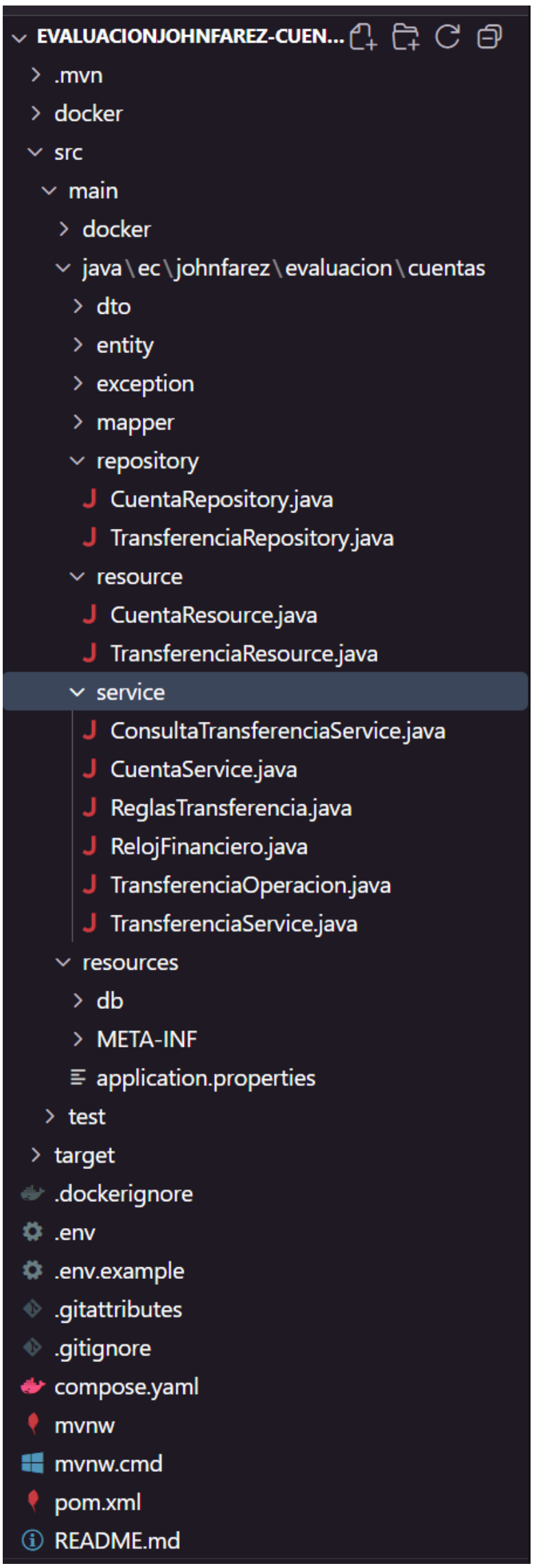
Capturas que acompañan la respuesta

1. Arquitectura en capas Backend Cuentas
2. Manejo centralizado de excepciones

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-08: Arquitectura y separación de responsabilidades

Captura 1. Arquitectura en capas Backend Cuentas



Paquetes y clases separados por Resource, Service, Repository, Entity, DTO, Mapper y Exception.

CA-08: Arquitectura y separación de responsabilidades

Captura 2. Manejo centralizado de excepciones

```java
@Provider
public class ApiExceptionMapper implements ExceptionMapper<Exception> {
    private static final Logger LOG = Logger.getLogger(ApiExceptionMapper.class);

    @Override
    public Response toResponse(Exception exception) {
        if (exception instanceof OperacionFinancieraException financiera)
            return Response.status(financiera.status()).type(MediaType.
            APPLICATION_JSON)
                .entity(new ErrorResponse(financiera.codigo(), financiera.
                getMessage(), List.of(),
                    financiera.montoRestanteDisponible())).build();
        if (exception instanceof CuentaDuplicadaException)
            return error(409, "CUENTA_DUPLICADA", exception.getMessage(), List.of
            ());
        if (exception instanceof WebApplicationException web) {
            int status = web.getResponse().getStatus();
            return error(status, "HTTP_" + status,
                status == 400 ? "El cuerpo de la solicitud no es valido" : "La
                solicitud no pudo procesarse", List.of());
        }
        LOG.error("Error no controlado al procesar cuentas", exception);
        return error(500, "ERROR_INTERNO", "No se pudo procesar la solicitud",
        List.of());
    }

    static Response error(int status, String codigo, String mensaje,
    List<ErrorCampo> campos) {
        return Response.status(status).type(MediaType.APPLICATION_JSON)
            .entity(new ErrorResponse(codigo, mensaje, campos)).build();
    }
}
```

ApiExceptionMapper con @Provider; conversión centralizada de excepciones a HTTP y ErrorResponse.

CA-09: Calidad del código y buenas prácticas

Requisito del documento oficial

Criterio: El código debe seguir buenas prácticas y estándares de codificación: nombrado descriptivo en inglés o español (consistente), principios SOLID, manejo centralizado de excepciones (ExceptionMapper en Quarkus), uso adecuado de inyección de dependencias (@Inject), uso de DTOs para requests/responses (no exponer entidades JPA directamente), y anotaciones JPA/Hibernate correctas.

Respuesta al criterio

La estructura separa CrearCuentaRequest, CrearTransferenciaRequest y DTOs de respuesta de las entidades Cuenta y Transferencia. Incluye CuentaMapper, TransferenciaMapper y varios ExceptionMapper. La captura de ApiExceptionMapper en CA-08 muestra @Provider, ExceptionMapper<Exception>, manejo de excepciones financieras y respuestas de error uniformes.

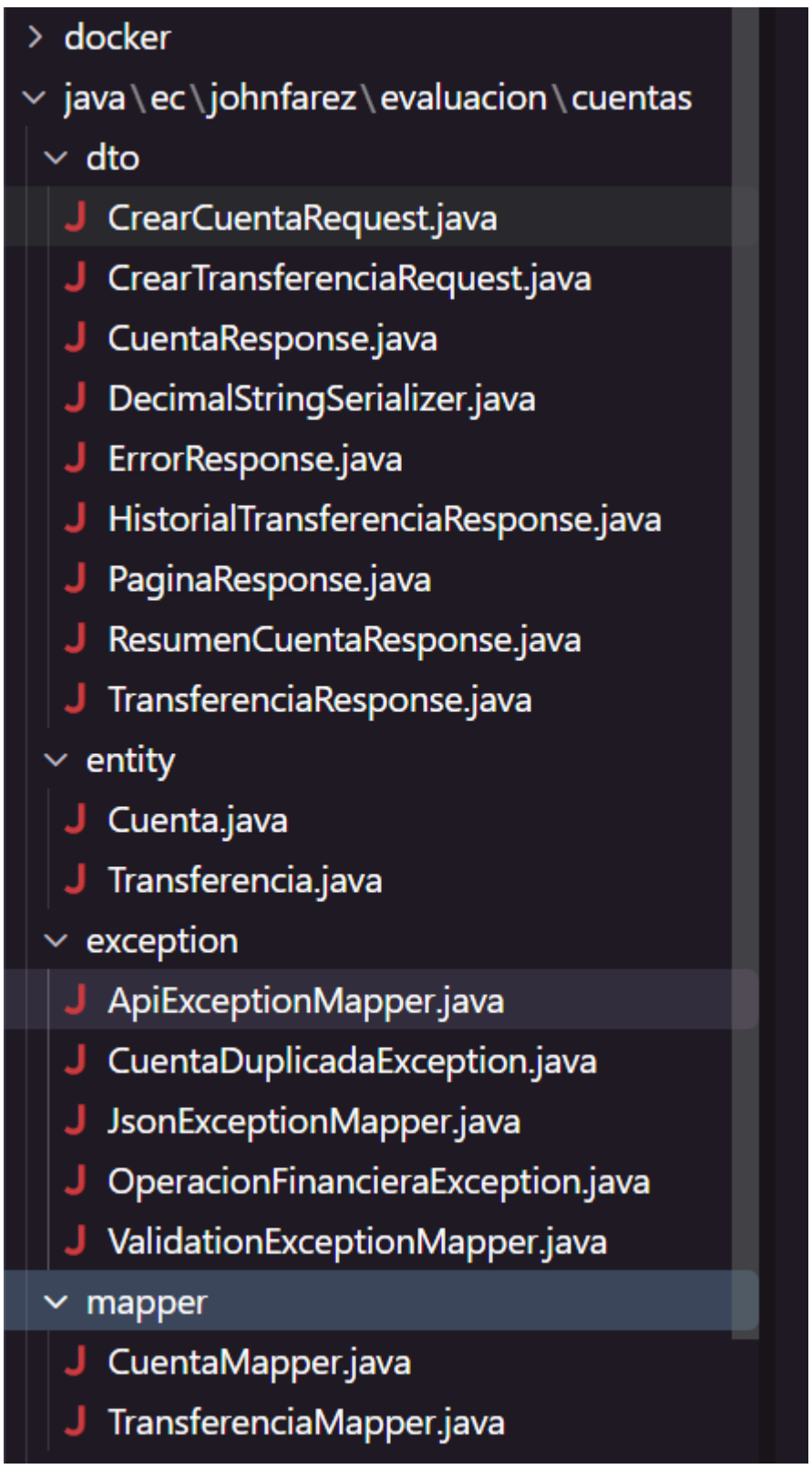
Capturas que acompañan la respuesta

1. Estructura y buenas practicas Java

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-09: Calidad del código y buenas prácticas

Captura 1. Estructura y buenas practicas Java



DTOs de entrada/salida, entidades, mappers y excepciones en paquetes independientes.

CA-10: Documentación (Swagger + README)

Requisito del documento oficial

Criterio: La API debe estar documentada con Swagger/OpenAPI incluyendo: descripción de cada endpoint, parámetros de entrada/salida, códigos de respuesta posibles y ejemplos. El repositorio debe incluir un archivo README.md con instrucciones claras para: configuración de la base de datos (script de creación o instrucciones), compilación del proyecto backend (mvn o gradle), ejecución del backend y frontend, y URL de acceso a la documentación Swagger.

Respuesta al criterio

La carpeta CA-010 muestra META-INF/openapi.yaml. Las capturas de CA-01, CA-02 y CA-05 muestran Swagger con rutas, parámetros, cuerpos, descripciones, códigos y ejemplos, además de ejecuciones reales. El README principal proporcionado y los README de backend y frontend revisados incluyen configuración de SQL Server, bootstrap, Flyway, compilación, arranque y URL de Swagger.

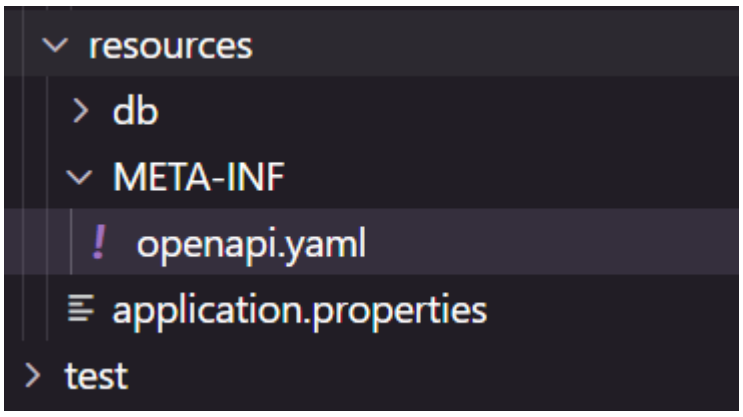
Capturas que acompañan la respuesta

1. Documentación

La respuesta describe la información visible en las capturas entregadas. Los ejemplos de Swagger describen el contrato de la API; las secciones Server response muestran los resultados de las solicitudes ejecutadas.

CA-10: Documentación (Swagger + README)

Captura 1. Documentación



Archivo resources/META-INF/openapi.yaml visible; la carpeta original se denomina CA-010.